TU WIEN Informatics

SAT-basierte Optimierung von oktolinearen Metrokarten

DIPLOMARBEIT

zur Erlangung des akademischen Grades

Diplom-Ingenieurin

im Rahmen des Studiums

Logic and Computation

eingereicht von

Samantha Fuchs, BSc
Matrikelnummer 01633020

an der Fakultät für Informatik

der Technischen Universität Wien

Betreuung: Univ.Prof. Dipl.-Inform. Dr.rer.nat. Martin Nöllenburg
Mitwirkung: Univ.Ass. Dipl.-Ing. Soeren Terziadis

Wien, 25. August 2022

Samantha Fuchs	Martin Nöllenburg

TU WIEN Informatics

SAT-based Optimization of Octolinear Metro Map Layouts

DIPLOMA THESIS

submitted in partial fulfillment of the requirements for the degree of

Diplom-Ingenieurin

in

Logic and Computation

by

Samantha Fuchs, BSc
Registration Number 01633020

to the Faculty of Informatics

at the TU Wien

Advisor: Univ.Prof. Dipl.-Inform. Dr.rer.nat. Martin Nöllenburg
Assistance: Univ.Ass. Dipl.-Ing. Soeren Terziadis

Vienna, 25th August, 2022

Samantha Fuchs	Martin Nöllenburg

Technische Universität Wien
A-1040 Wien ▪ Karlsplatz 13 ▪ Tel. +43-1-58801-0 ▪ www.tuwien.at

Erklärung zur Verfassung der Arbeit

Samantha Fuchs, BSc

Hiermit erkläre ich, dass ich diese Arbeit selbständig verfasst habe, dass ich die verwendeten Quellen und Hilfsmittel vollständig angegeben habe und dass ich die Stellen der Arbeit – einschließlich Tabellen, Karten und Abbildungen –, die anderen Werken oder dem Internet im Wortlaut oder dem Sinn nach entnommen sind, auf jeden Fall unter Angabe der Quelle als Entlehnung kenntlich gemacht habe.

Wien, 25. August 2022

Samantha Fuchs

Danksagung

Ich möchte mich an dieser Stelle bei einigen Personen bedanken, ohne die diese Arbeit garnicht oder nicht in dieser Form entstanden wäre.

Angefangen bei meinem Betreuerteam aus Prof. Martin Nöllenburg und Soeren Terziadis die mein Interesse für Metro Maps geweckt haben und mich mit spannenden Disskussionen und Anregungen zum Thema auf Trapp gehalten haben. Als Teil der Algorithms and Complexity Research Group haben Sie mir auch die Möglichkeit verschafft entsprechende Hardware für die durchführung der nötigen Experimente mitzubenützen. Dafür bin ich ihnen und der ganzen Gruppe sehr dankbar.

Mein Dank gebührt außerdem der ganzen Technischen Universität Wien sowohl für die Chance mein Bachelor und Masterstudium ohne größere Hürden abzuschließen als auch für die Möglichkeit ein individuelles Forschungsthema auswählen zu können. Zusätzlich habe ich durch die Universität die Unterstützung bekommen um am Schematic Mapping Workshop 2022 an der Ruhr-Universität Bochum teilzunehmen. Durch diese Teilnahme habe ich noch mehr interessanten Input und Vorschläge für die Weiterentwicklung meines Projekts bekommen.

Des weiteren hat Nikolaj Bjørner von Microsoft Corporation speziellen Dank verdient. Als Entwickler des z3 SMT Solvers hat er unseren Vorschlag einer Feature Erweiterung herzlich angenommen und sofort umgesetzt, was bei weitem nicht selbstverständlich war.

Abschließend möchte ich mich bei meiner Familie und meinen Freunden für die emotionale und mentale Unterstützung bedanken. Zu nennen wären da unter anderem Dorit Fuchs, Thomas Janka, Katharina Joksch und Maximilian Marschall, die neben ihrer Lektoratstätigkeit auch kreative Denkanstöße geliefert haben und immer ein offenes Ohr hatten.

Acknowledgements

At this point I want to acknowledge the help of several people, without whom this thesis would not be the same or not even exist.

I want to begin with my advisor Prof. Martin Nöllenburg and his assistant Soeren Terziadis for raising my interest about the topic of metro maps. We had interesting discussions that included quite a lot suggestions about the given topic. They gave me the opportunity to use hardware provided by the Algorithms and Complexity group needed for experiments.

I am also thankful to the Technical University of Vienna for the possibility to complete my Bachelor and Masters degree without significant problems as well as for having the chance to work on an individual research topic. Additionally the university funded my participation at the Schematic Mapping Workshop 2022 at Ruhr-University of Bochum. There I got interesting feedback and additional input for developing my project.

I also have to say thank you to Nikolaj Bjørner from Microsoft Corporation. He is developer of the z3 SMT solver and easily implemented our feature request without hesitation, which helped us a lot with the experiments.

At last I want to thank my friends and family for supporting me emotional and mental during the time I worked on this thesis. Especially I want to mention Dorit Fuchs, Thomas Janka, Katharina Joksch and Maximilian Marschall. Beside proofreading several sections of the thesis they gave me creative thoughts and always had an open ear.

Kurzfassung

U-Bahn-Karten werden jeden Tag von Millionen von Menschen auf der ganzen Welt gesehen und verwendet. Die Erstellung der besten Zeichnung eines U-Bahn-Netzes ist eine komplexe und zeitaufwändige Aufgabe. Tatsächlich ist das Metrokarten Problem NP-schwer.

Dieses Problem kann auf verschiedene Arten formalisiert werden. Wir verwenden eine Definition aus der Literatur. Eine solche Formalisierung ermöglicht es uns Algorithmen zu entwickeln um automatisiert eine Lösung für das Problem zu finden. So wurden in der Literatur Mixed Integer Programs (MIP) auf der Basis dieser Formulierung entwickelt, die es ermöglichen *optimale* Metrokarten zu berechnen.

Für andere Graph Drawing Probleme haben sich auch Modellierungen mithilfe von SAT und SAT modulo Theory (SMT) als nützlich und in manchen Fällen sogar als effizienter als MIP Formulierungen erwiesen. Wir präsentieren die erste Formulierung des Metrokarten Problems mithilfe von SAT und SMT und vergleichen die Geschwindigkeit und Qualität der produzierten Lösungen mit der existierenden MIP Formulierung.

Um eine Lösung mit SAT zu finden, übersetzen wir die Bedingungen aus einem bestehenden MIP in SAT. Die Herausforderung besteht hauptsächlich darin die Constraints der MIP Formulierung effizient in ein System zu übersetzen in welchem statt ganzzahligen und reelwertigen Variablen nur Binärvariablen verwendet werden können. Die Übersetzung nach SMT ist einfacher, weil hier nicht nur boolesche Variablen akzeptiert werden, sondern auch numerische Werte oder sogar andere Datentypen.

Unser Datensatz besteht aus fünf verschiedenen realen Instanzen: den U-Bahn-Netzen der Städte Wien, Karlsruhe, Lissabon, Montreal und Washington. Wir analysieren die Effizienz unseres Ansatzes mit Experimenten auf diesen Instanzen und vergleichen die Laufzeit, aber auch die Qualität nicht optimaler Lösungen nach einem bestimmten Timeout.

Unsere Ergebnisse legen nahe, dass MIP für das schematische Metrokarten Problem schneller ist. Die Unterschiede zwischen SAT und SMT variieren abhängig von der Instanzgröße, wobei SAT bei kleineren Instanzen besser abschneidet während SMT bessere Laufzeiten auf größeren Instanzen erzielt. Dies deutet darauf hin, dass bei SMT ein größerer Initialisierungsaufwand besteht.

Abstract

Metro maps are seen and used by millions of people around the globe every day. The process of creating the most optimized drawing of a metro network is a complex and time-consuming task. In fact, the metro map problem is NP-hard.

This problem can be formalized in several ways. We use some definition from the literature. Such a formalization enables us to develop algorithms to automatically find a solution to the problem. There are already Mixed Integer Programs (MIP) based on this formulation, which allows to calculate *optimal* metro maps.

For other graph drawing problems, modeling using SAT and SAT modulo theory (SMT) has also proven to be useful and in some cases even more efficient than MIP formulations. We present the first formulation of the metro map problem using SAT and SMT and compare runtimes and quality of the solutions produced with the existing MIP formulation.

To find a solution with SAT we translate the constraints of an existing MIP definition into SAT. The main challenge is to efficiently translate the constraints of the MIP formulation into a system in which only binary variables can be used instead of integer and real-valued variables. The translation to SMT is easier because it not only accepts boolean variables but also integer values or even other data types.

Our dataset consists of five different real world instances: the metro networks of the cities Vienna, Karlsruhe, Lisbon, Montreal and Washington. We analyze the efficiency of our approach with experiments on these instances and compare the runtime but also the quality of non-optimal solutions after a given timeout.

As a result, we found out that MIP is faster for the schematic metro map problem. The results on SAT and SMT are different between instances, depending on their size with SAT outperforming SMT on smaller instances, while SMT achieves better runtimes on larger instances. This indicates that there is a larger initialization overhead needed for SMT.

Contents

CHAPTER 1

Introduction

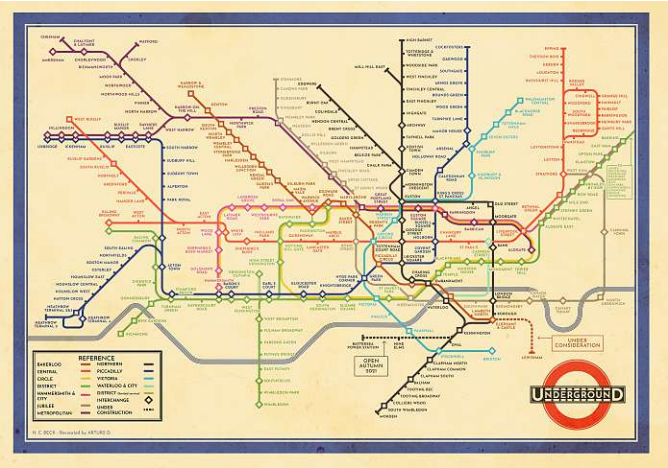


Figure 1.1: Harry Becks London Tube Map from 1933 [Bec33].

Automated graph drawing is a widely researched topic and has a variety of use cases. Several examples should be mentioned, like the generation of workflow maps or family trees. One important use case is the design and generation of transit maps in general. Their main purpose is to display geographical data like public transportation networks. These transit maps can be represented as schematic maps like Luteberget et al. [LCJ19] do it for railway networks. Other examples for schematic maps are linear catograms by Van Dijk and Lutz [vDL18] or Kelp diagrams by Dinkla et al. [DvKSW12]. Another prominent example for schematic transit maps are schematic metro maps [WNT+20] that date back to Harry Becks london tube map from 1933 in Figure 1.1. Transit maps or schematic metro maps in particular are also used to represent non-spatial data as shown with metro map style set visualization by Jacobsen et al. [JWKN21]. In Figure 1.2 we can see a metro map style visualization of a Simpsons dataset.

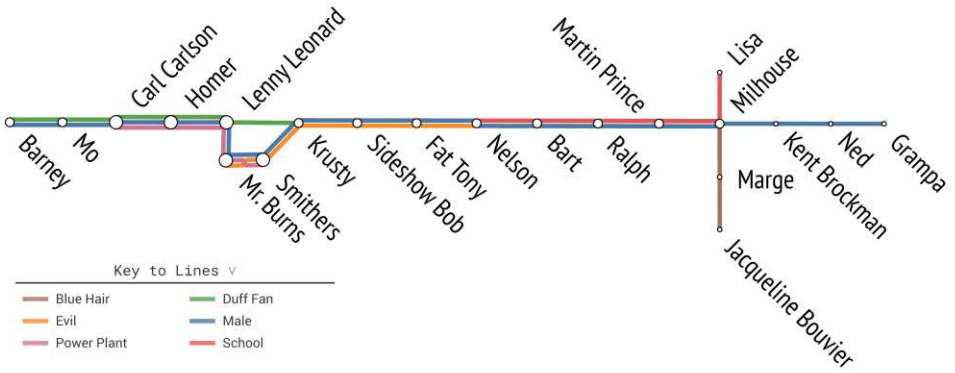


Figure 1.2: The Simpsons data sets visualized in metro map style [JWKN21].

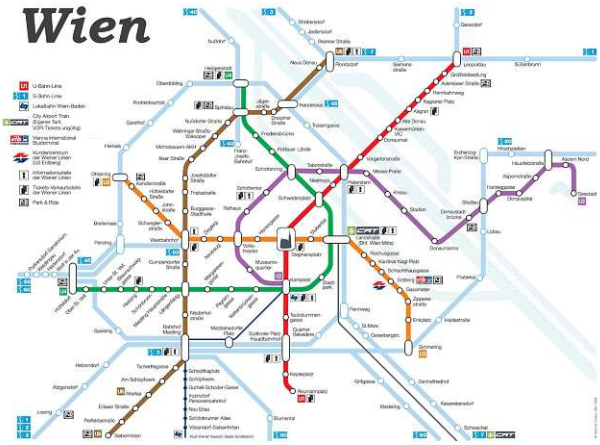
Independent of the maps that should be produced, the desired drawings can have several conventions that have to be satisfied in all cases, for example to restrict the amount of edge directions, to force grid placement or to be crossing free. Aesthetic criteria can be area minimization, uniform edge length, crossing minimization, or any other criteria that should be fulfilled as well as possible, but do not have to be perfect in each solution.

The use case we will work with in this thesis is the design of schematic metro maps [WNT+20] as known from metro networks that are scattered all over the world. In all of these places, metro maps are needed inside the metros themselves as well as in and around metro stations. This makes metro maps to often-used drawings, which are simplified maps, that emphasize network connectivity at the expense of spatial accuracy. There exist several sets of conventions that can be used to generate these simplified metro maps. Schematic metro maps as we will define them in more detail in Chapter 2 restrict the edge directions to four different directions. This criteria is called octolinearity. Besides that, we force grid placement and want to preserve the relative order of connections around each station. The aesthetic criteria are area minimization and minimization of bends. In addition, the edge directions should represent reality as good as possible.

The construction of schematic metro maps is time consuming and complicated, hence an automatic tool can help designers by significantly speeding up the creation of metro maps. In extension an automatic tool enables the exploration of more different designs when using different algorithms with several design conventions [WNT+20].

Many graph drawing problems that optimize some criteria lead to NP-hard problems. Examples for this are bend minimization in orthogonal drawings [GT01] and the area minimization in compact drawings [YDG+16]. Similarly, the metro map layout problem has been proven NP-hard, by proving that the bend minimization itself is NP-hard in schematic metro maps [Nöl05]. This means we cannot expect an efficient algorithm to solve this problem.

For this reason the first approaches to solve metro maps are heuristics. In this area we have found two main techniques that do not guarantee the core defining aspect of



(a) Metro map of Vienna.

(b) Metro map of Washington.

(c) Metro map of Sydney.

Figure 1.3: Schematic metro maps used for real world metro networks.

octolinearity. The first one is Least-Squares Optimization used by Wang and Chi [WC11]. The second technique is used by different authors, it is Path-Based Schematization and was used by Merrick and Gudmundsson [MG06] and by Dwyer et al. [DHM08]. One common property of these approaches is that, while they are fast, they do not guarantee octolinearity. Another method, which can guarantee octolinearity is the approach of Bast et al. [BBS20, BBS21], which presents a new model for the layout problem as well as a fast heuristic approach. Chivers and Rodgers [CR14] introduced a force-based technique . It combines two separate steps: A spring embedder to distribute stations followed by a magnetic force field to get an octolinear layout.

Even if the schematic metro map problem is NP-hard, there are already exact solutions to this problem with combinatorial methods. One example is the second contribution of Bast et al. [BBS20, BBS21]: a Mixed-Integer Programming (MIP) formulation beside their heuristic approach that gives optimal results. This shows that an optimal solution can feasibly be computed on reasonably sized real world instances, like the commonly used benchmark maps of Sydney, Vienna, Washington and others (the official metro maps, of these cities are shown in Figure 1.3).

Nöllenburg and Wolff [NW10] use MIP to solve the schematic metro map problem optimally in reasonable time on real world instances, but it is still relatively slow and can take many hours. Nickel and Nöllenburg [NN20] extended this approach to any user-defined set of directions (for example orthogonal or hexalinear drawings or not evenly distributed directions).

It is quite common for NP-hard problems to be modeled as a Mixed-Integer Program (MIP) or a SAT solving problem (SAT). This has the advantage that we are able to encode conventions and aesthetic criteria as constraints and use existing solvers to compute solutions. SAT and MIP encodings have shown discrepancies in runtime for other graph drawing problems, for example for the already mentioned compact layout drawings [YDG+16]. A similar problem to schematic metro maps is the railway schematics problem, solved with SAT and MIP by Luteberget et al. [LCJ19]. The exact reason for the runtime discrepancies is unclear. This suggests it is interesting to investigate the viability of a SAT encoding for the metro map layout problem as well. To the best of our knowledge, the metro map problem has not yet been approached from a SAT point of view.

There already exists work about general translations from MIP to SAT, but these procedures cannot translate any MIP into SAT, only specific forms of programs. An example is the work of Eén and Sörensson [ES06], which translates pseudo-Boolean MIP constraints into SAT. As far as we know, there is no solution for translating any restriction-less MIP into SAT and therefore also no solution to translate existing MIP formulations of the schematic metro map problem.

The main goal of this thesis is to formulate a SAT encoding for the schematic metro map problem. Using unary integer encoding to encode the coordinates of stations/nodes we transferred the MIP constraints by Nöllenburg and Wolff [NW10] into SAT clauses.

Additionally we used an extension to SAT, the Satisfiability Modulo Theory (SMT), to translate the constraints we gained from the MIP encoding directly, without the necessity of unary integer encoding. This is possible, because SMT solvers extend the functionality of SAT solvers in a way that allows the usage of integer values and the use of constraints over integers.

The following chapter contains the preliminaries to give some relevant definitions, that are needed as basic concepts for this thesis. The third and fourth chapter give the detailed SAT and SMT constraints that are generated by our encoding.

In the fifth chapter we give a comparison of the runtimes of the SAT and SMT models. Evaluating on common test instances used in metro map literature enables us to compare these results also against the already existing MIP formulations. This will give an insight about the effectiveness of SAT and SMT encodings for automated generation of schematic metro maps.

The last chapter of this thesis gives a summary about our work and results as well as an outlook about further possible work on schematic metro maps in combination with SAT or SMT.

CHAPTER 2

Preliminaries

In this section we introduce some basic concepts and the relevant notations, which will be used throughout the thesis. Concepts that are of interest only at an isolated part of the thesis can be defined and explained locally at that point. The first concept is about graphs and their basic properties. After that we define schematic metro maps. Then some concepts that are used as speed up methods are described as preprocessing steps.

2.1 Graph theory

A graph $G = (V, E)$ is defined by a set of vertices V and a set of edges $E \subseteq V^2$. To distinguish between vertices of two different graphs G_1 and G_2 one can also use the notation of $V(G_1)$ and $V(G_2)$ for these two vertex sets. The same holds for the edge sets $E(G_1)$ and $E(G_2)$. An edge $(u, v) \in E$ is a none ordered pair of vertices $u, v \in V$, that represents a connection between those two vertices. Note that by this definition $(u, v) = (v, u)$ holds. A small instance is given in the following example.

Example 1. Graph $G_1 = (V_1, E_1)$
$V_1 = \{1, 2, 3\}$
$E_1 = \{(1, 2), (1, 3), (2, 3)\}$

Two vertices u, v that are connected by an edge $e = (u, v)$ are called adjacent. Similarly an edge (u, v) is called incident to the vertices u and v. Adjacent vertices u and v are also called neighbors. The neighborhood $N(v)$ of a vertex v is the set of all neighbors of v. The degree $deg(v)$ of a vertex V is the number of neighbors of v, hence it is the size of its neighborhood: $deg(v) = |N(v)|$.

The graph K_n is called the complete graph with n vertices. It has an edge between every pair of vertices. The graph in Example 1 matches the graph K_3. In Example 2 the complete graph K_4 is defined.

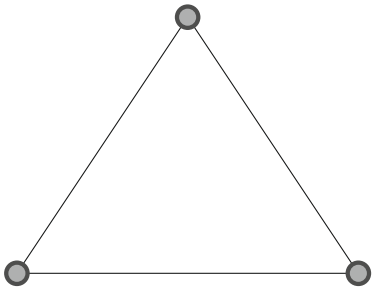


Figure 2.1: Drawing of graph G_1.

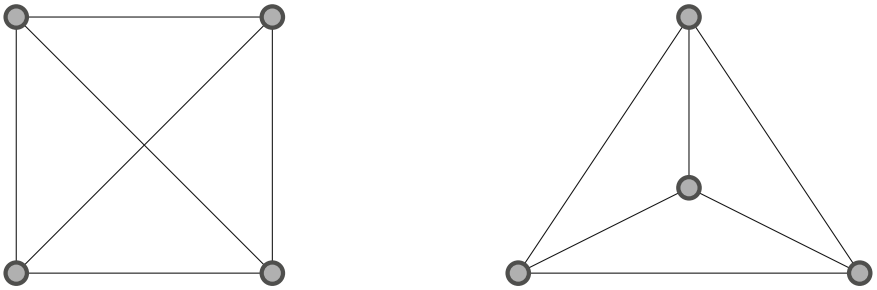
Example 2. Complete graph K_4

$V(K_4) = \{1, 2, 3, 4\}$

$E(K_4) = \{(1, 2), (1, 3), (1, 4), (2, 3), (2, 4), (3, 4)\}$

A graph can be drawn using dots or disks or any other representation of a point for each station and lines for each edge. A drawing for Example 1 is given in Figure 2.1.

In a more formal definition, a drawing $\Gamma(G)$ of a graph $G = (V, E)$ is an embedding into $\mathbb{R}^2$ that maps vertices in V to points in $\mathbb{R}^2$ and edges in E to curves in $\mathbb{R}^2$ [Pat13, Tam13]. With this definition as substantiation drawing and embedding can be be used as synonyms. Drawings that map edges to straight lines in $\mathbb{R}^2$ are called straight-line drawings.

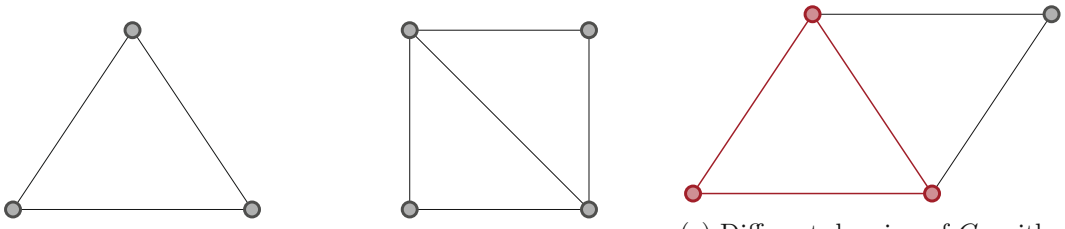


(a) Drawing of K_4 containing one crossing (b) Crossing free, planar drawing of K_4

Figure 2.2: Drawings of the planar complete graph K_4.

A drawing $\Gamma(G)$ of a graph G is planar, if it has no edge crossings. An edge crossing is an intersection of two distinct edges, that do not have common endpoints. A graph G is a planar graph, if there exists at least one planar (straight-line) drawing of G Note that if there exists any planar drawing of a graph G, then there also exists a planar straight-line drawing of G [Vis13, SR34, Wag36, Fár48, Ste51]. Conversely not all drawings of a planar graph are necessarily crossing-free. In Figure 2.2 two different straight-line drawings for the same planar graph K_4 are given as illustration of this fact, the first one has crossings and the second one is crossing-free, hence the first one is non-planar, but the second one is planar.

In this thesis we only consider straight-line drawings when referring to drawings and omit the designation “straight-line”. We use the opportunity to introduce dummy vertices in some cases to represent edges with line bends without violating the definition of straight-line drawings, but that is stated explicitly if we do so.



(a) G_1 (b) G_2 (c) Different drawing of G_2 with subgraph G_1 highlighted.

Figure 2.3: Different drawings of the graphs G_1 and G_2 from Example 1 and Example 3. The subgraph G_1 is highlighted in red.

A graph $G = (V, E)$ is a subgraph of the graph $G' = (V', E')$ (written $G \subseteq G'$), if $V \subseteq V'$ and $E \subseteq E'$ hold. The graph G_1 from Example 1 is a subgraph of G_2 from Example 3 (short $G_1 \subseteq G_2$). This is illustrated in Figure 2.3.

Example 3. Graph $G_2 = (V_2, E_2)$

$V_1 = \{1, 2, 3, 4\}$

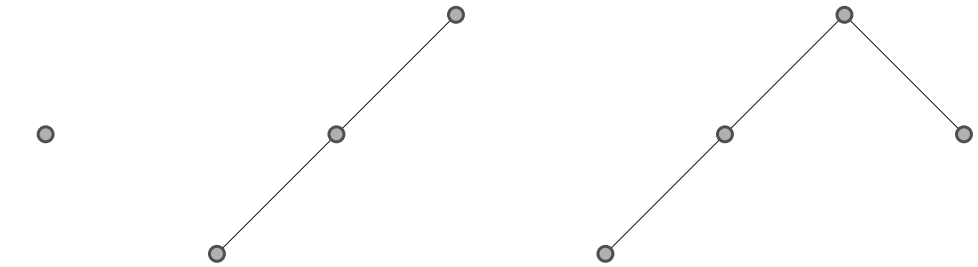
$E_1 = \{(1, 2), (1, 3), (1, 4), (2, 3), (3, 4)\}$

The union of two graphs $G = (V, E)$ and $G' = (V', E')$ is the graph $G \cup G' = (V \cup V', E \cup E')$.

A path of length n is a graph $P(n)$ with n vertices. These vertices can be ordered into a sequence of the distinct vertices $v_1, v_2, \ldots, v_n \in V(P_n)$. The edges of $P(n)$ are $(v_1, v_2), (v_2, v_3), \ldots, (v_{n-2}, v_{n-1})(v_{n-1}, v_n)$. Examples of paths are given in Figure 2.4. Note that a path $P(1)$ of length $n = 1$ does not have any edges, but is also a path per definition. Adding the edge (v_n, v_1) to some path $P(n)$ transforms it into the cycle $C(n)$. The graph in Example 1 and Figure 2.1 is the cycle $C(3)$.

A line cover $\mathcal{L}$ of a graph G is a set of paths $P_1, P_2, \ldots, P_k$, that are all subgraphs of G, hence $P_i \subseteq G$ holds for each $1 \leq i \leq k$. Additionally the paths P_i together cover the whole graph G, i.e. $\bigcup_{i=1}^{k} P_i = G$ holds. The vertices and edges of the paths in a line cover do not have to be distinct, hence an edge or a vertex can be part of several paths in the line cover. In some definitions a line cover does not have to be a set of paths, but can be a set of subgraphs of G instead, which is a more general definition. In this thesis the more general definition is used to have less restrictions for the input.

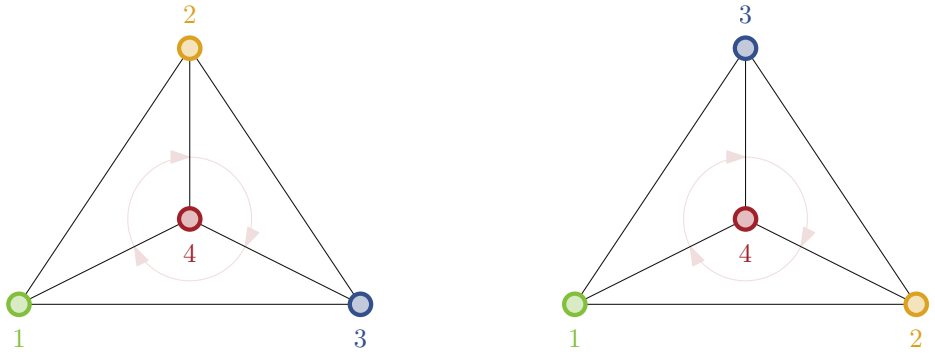
The neighbors $u_i \in N(v)$ of a vertex v can be ordered in a drawing $\Gamma(G)$ by the order in which the edges (v, u_i) are encountered when walking clockwise on the boundary of a small circle around v Using this procedure one can generate $|N(v)|$ different orderings,



(a) $P(1)$ (b) $P(3)$ (c) $P(4)$

Figure 2.4: Drawings of paths with different length.

each starting with a different vertex of $N(v)$. We consider these different orderings as equivalent.



(a) $\Gamma_1(K_4)$ (b) $\Gamma_2(K_4)$

Figure 2.5: Different planar drawings of the complete graph K_4, indicating the circular order of vertex 4.

The drawings given in Figure 2.5 show two different planar drawings of the complete graph K_4. Observing vertex 4 in drawing $\Gamma_1(K_4)$ we can gather the circular orders given in Table 2.1.

$\Gamma_1(K_4)$	$\Gamma_2(K_4)$
(1, 2, 3)	(1, 3, 2)
(2, 3, 1)	(3, 2, 1)
(3, 1, 2)	(2, 1, 3)

Table 2.1: Circular orders of vertex 4 for drawings given in Figure 2.5. The orderings are equivalent within each column.

Note that both drawings in Figure 2.5 are drawings of the same graph and the orderings given in Table 2.1 are all circular orderings in clockwise direction for the vertex 4, but the orderings for drawing $\Gamma_1(K_4)$ differ from the orderings for drawing $\Gamma_2(K_4)$.

An embedding $\Gamma(G)$ of some graph $G = (V, E)$ has $|V|$ vertices and $|E|$ edges. It also contains faces, that are regions in the plane bounded by cycles. Additionally it has the infinite unbounded outer face, which is the region that is outside of the graph. We denote the set of faces by $F(\Gamma(G))$, if we have to distinguish between different graphs or embeddings. We will omit $\Gamma(G)$ and simply write F if it is clear from context. Recap the graph of Example 3, it has the outer face f_0 and 2 bounded faces f_1 and f_2 as illustrated in Figure 2.6.

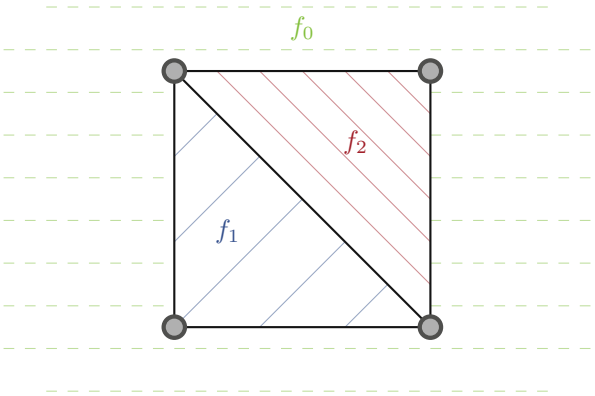


Figure 2.6: Drawing of graph G_2 with highlighted faces.

The number of faces $|F|$ of a planar graph is bounded by Euler's formula.

Theorem 1 (Euler's formula [Eul58, Leg85])**.**

$$|F| + |V| - |E| = 2$$

In further consequence this shows that planar graphs are sparse. A maximal planar graph has only faces that are bounded by exactly 3 edges, hence $3|F| = 2|E|$. Adding an edge to these graphs would make them non planar. An example for a maximal planar graph is the complete graph K_4. Its planar drawing is given in Figure 2.2b. There we can see that each face is a triangle and also the outer face has exactly 3 edges. Note that this does not hold for complete graphs K_n with $n \geq 5$, because these graphs are not planar. Using Euler's formula we get $|E| = 3|V| - 6$ for maximal planar graphs. Deleting some edges from a maximal planar graph does not have an effect on the planarity property, hence we get Corollary 1.

Corollary 1.

$$|E| \leq 3|V| - 6$$

This means that a planar graph with n vertices can have at most $3n - 6$ edges. Vice versa a graph with n vertices and more than $3n - 6$ edges can not be planar.

2.2 Schematic metro map layout

A schematic map, is a drawing or in other words a representation of a graph or other graph structured data, that follows a set of design rules, that can vary by definition. We will work with the definition of Nöllenburg and Wolff [NW10] and recap it now.

The input of a schematic metro map consists of three things:

1. A graph $G = (V, E)$ built from the vertex set V and the edge set E.
2. A line cover $\mathcal{L}$ of the graph G.
3. An embedding $\Gamma_I(G)$ of the graph G.

This input is used to generate a schematic drawing $\Gamma(G)$, following the design rules defined in (H1) to (H4).

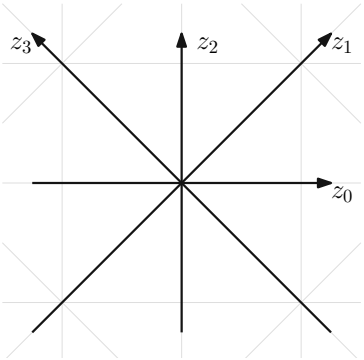


Figure 2.7: Octolinear directions.

(H1) Edge directions: The direction of an edge has to be out of a predefined set of directions. We use the octolinearity restrictions, hence the direction of an edge can be one of 4 different directions. These directions illustrated in Figure 2.7 are the following:

- z_0: 0° horizontal
- z_1: 45° diagonal
- z_2: 90° vertical
- z_3: 135° diagonal

Note that a consequence of this rule is, that the vertices of the input graph can have a degree of at most 8, hence $deg(v) \leq 8$ must hold for each vertex v, because otherwise the rule can not be satisfied. That can be proven with the pigeonhole principle given in Theorem 2: If some vertex v has degree $deg(v) > 8$, than the

$n = deg(v)$ edges to v's neighbor are the objects that have to be distributed over $m = 8$ directions and by the pigeonhole principle some direction would receive at least two edges.

Theorem 2 (Pigeonhole principle [Her64])**.** *If n objects are distributed over m places, and if $n > m$, then some place receives at least two objects.*

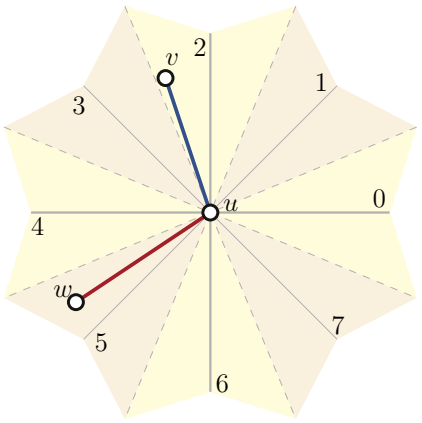


Figure 2.8: Octolinear sections. The blue edge (u, v) lies in section 2, the red edge (u, w) in section 5, because that is the nearest octolinear direction in each case.

As suggested by Nöllenburg and Wolff [NW10] and extended by Nickel and Nöllenburg [NN20] we restrict the possible directions for each edge a bit further. For the octolinear drawing we divide the output directions of each node into eight sections. These sections are divided exactly between the octolinear directions. With this division each input edge $e = (u, v)$ is assigned to the sector corresponding to its nearest octolinear direction that we call $sec_u(v)$. The division is illustrated in Figure 2.8 with two example edges. Note that for each edge $e = (u, v)$ we also define $sec_v(u) = sec_u(v) + 4 \mod 8$. The assignment of sections to each edge is a preprocessing step on the input, before automated generation of the embedding itself starts. That is the reason why these section values $sec_u(v)$ can be handled as constants when modeling the problem, since they only depend on the input, that is fixed during the generation of the embedding.

Now the suggested restriction is to only allow directions for an edge (u, v) in the output drawing, that have a maximal deviation to $sec_u(v)$. The default value of this deviation is $dev = 1$. From this definition we can generate a set $S(u, v)$ of allowed directions for each edge (u, v):

$$S(u, v) = \{i \mod 8 \mid sec_u(v) - dev \leq i \leq sec_u(v) + dev\}$$

(H2) Circular vertex order: The combinatorial embedding $\Gamma_I(G)$ should be preserved. This means, for each vertex v, the circular order of its neighbors has to be the same in the output drawing, as it was in the input embedding $\Gamma_I(G)$.

(H3) Minimum edge length: If a minimum edge length $L_{min}(e) = L_{min}(u, v)$ is given for an edge $e = (u, v)$, then it has to be ensured, that the distance between the vertices u and v is at least $L_{min}(u, v)$ in the resulting drawing. The default value for each edge e without a given minimum distance is $L_{min}(e) = 1$.

(H4) Planarity and edge spacing: To get a planar drawing, two edges that are not adjacent to the same vertex, should not cross. Two non crossing edges $e = (u, v)$ and $e' = (u', v')$ can be separated by a line as explained by Nöllenburg and Wolff [NW10]. In an octolinear drawing a separating line can be found that is octolinear as well. As we work with integer coordinates, it is clear that a separation like this results in a distance of at least $d_{min} = 1$. If a higher distance d_{min} is given for edge separation, then the distance between edges should be correspondingly larger.

From these definitions we get some optional input parameters for the schematic metro map layout problem:

A. The minimum edge length $L_{min}(e) = L_{min}(u, v)$ for each edge $e = (u, v)$.

B. The minimum distance d_{min} to separate edges.

C. The allowed deviation *dev* for edge directions.

Using the design rules $(H1)$ to $(H4)$ as hard constraints we can define the schematic metro map layout problem. The output embedding $\Gamma(G)$ is defined by the x and y coordinates for each vertex v.

Problem 1 (Schematic metro map layout problem)**.**

Input:	*A planar graph $G = (V, E)$ with maximum degree 8, a line cover L of G and an embedding $\Gamma_I(G)$ of G.*
Optional:	*A minimum edge length $L_{min}(e)$ for each edge e, the minimal distance d_{min} and the deviation dev.*
Question:	*Find an embedding $\Gamma(G)$ that satisfies the hard constraints $(H1)$ to $(H4)$.*
Output:	*An embedding $\Gamma(G)$ or* Infeasible *if this is not possible.*

The design rules defined in $(H1)$ to $(H4)$ are restrictions that the schematic map has to strictly follow in any occasion. Beside these hard constraints there is another set of design rules $(S1)$ to $(S3)$, that has to be satisfied as good as possible.

(S1) Line bends: The line bends on each metro line should be minimal. That's because a drawing seems less complex with less bends as discussed in the Survey of Wu et al. [WNT+20] and ones sight can follow a straight line better, than a

bend line. If there is a line bend in the output embedding, then the angle of this bend should be as large as possible. The angle measurement of line bends works as follows: take the angles in clockwise and in counter clockwise direction. The smaller one is the angle of the line bend. Hence the line bends are at most 180° each.

(S2) Relative positions: On the input embedding each edge (u, v) has some direction $sec_u(v)$ as described in Figure 2.8. This direction should be preserved as well as possible.

(S3) Compactness: The overall edge length should be minimal. This is achieved by summing up all edge lengths and minimizing the resulting sum.

To balance these design rules, we introduce a vector of integer weights $w = (f_1, f_2, f_3)$ containing one weight for each rule. The higher some weight f_i, the more important is the corresponding rule (Si). Its default value is $w = (1, 1, 1)$. This vector is another optional input parameter for the schematic metro map problem:

D. Vector of integer weights $w = (f_1, f_2, f_3)$ to balance importance of soft constraints $(S1), (S2)$ and $(S3)$.

Using the design rules $(S1)$ to $(S3)$ as soft constraints we can extend the schematic metro map layout problem to the schematic metro map problem that we will work on in this thesis.

Problem 2 (Schematic metro map problem)**.**

Input:	*A planar graph $G = (V, E)$ with maximum degree 8, a line cover L of G and an embedding $\Gamma_I(G)$ of G.*
Optional:	*A minimum edge length $L_{min}(e)$ for each edge e, the minimal distance d_{min} and the deviation dev.* *Additionally weights for each soft constraint (f_1, f_2, f_3).*
Question:	*Find an embedding $\Gamma(G)$ that satisfies the hard constraints $(H1)$ to $(H4)$ and optimizes the soft constraints $(S1)$ to $(S3)$ w.r.t the weights (f_1, f_2, f_3).*
Output:	*An embedding $\Gamma(G)$ or* INFEASIBLE *if this is not possible.*

Note that the output embedding $\Gamma(G)$ is still defined by the x and y coordinates for each vertex v as it was at the schematic mapping problem.

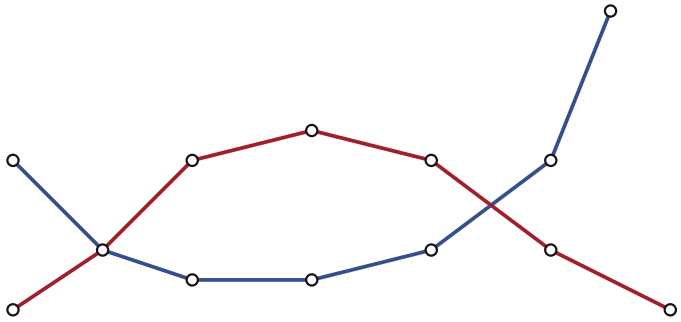
2.3 Preprocessing

Our models allow the use of common preprocessing steps to manipulate the input. This is done to satisfy the criteria for the input, for example to get planar graphs and also

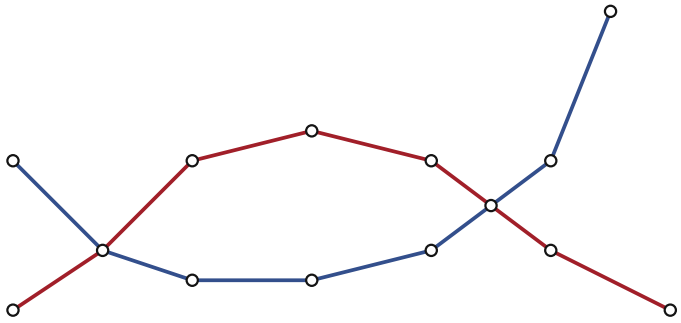
to reduce input sizes and by this means to reduce runtimes. To use these preprocessing techniques and produce a correct output afterwards it is necessary to revert the changes in postprocessing steps in reverse order. To do so one has to keep track of the changes and store them during the model calculations.

In this section we describe the preprocessing steps used for our experiments. The first one is planarization to transform non planar embeddings into planar ones. This is only necessary for input graphs with non planar embedding. The second one is the two degree heuristic to reduce the size of the input graph.

2.3.1 Planarization



(a) Planar input graph, with non planar embedding.



(b) Transformation with the planarization step.

Figure 2.9: Planar input graph, with non planar embedding and its transformation into a new graph with a planar embedding.

In Figure 2.9a the drawing of an example graph is given, that has a crossing, hence it is not a planar drawing. The goal is now to keep the embedding, hence keep the faces as they are. To do so, we add a station s exactly on the crossing of the edges $e_1 = (u_1, v_1)$ and $e_2 = (u_3, v_2)$. We also replace both edges by the split equivalents $e_{ia} = (u_i, s)$ and $e_{ib} = (s, v_i)$. This transformation is shown in Figure 2.9b.

2.3.2 Two degree heuristic

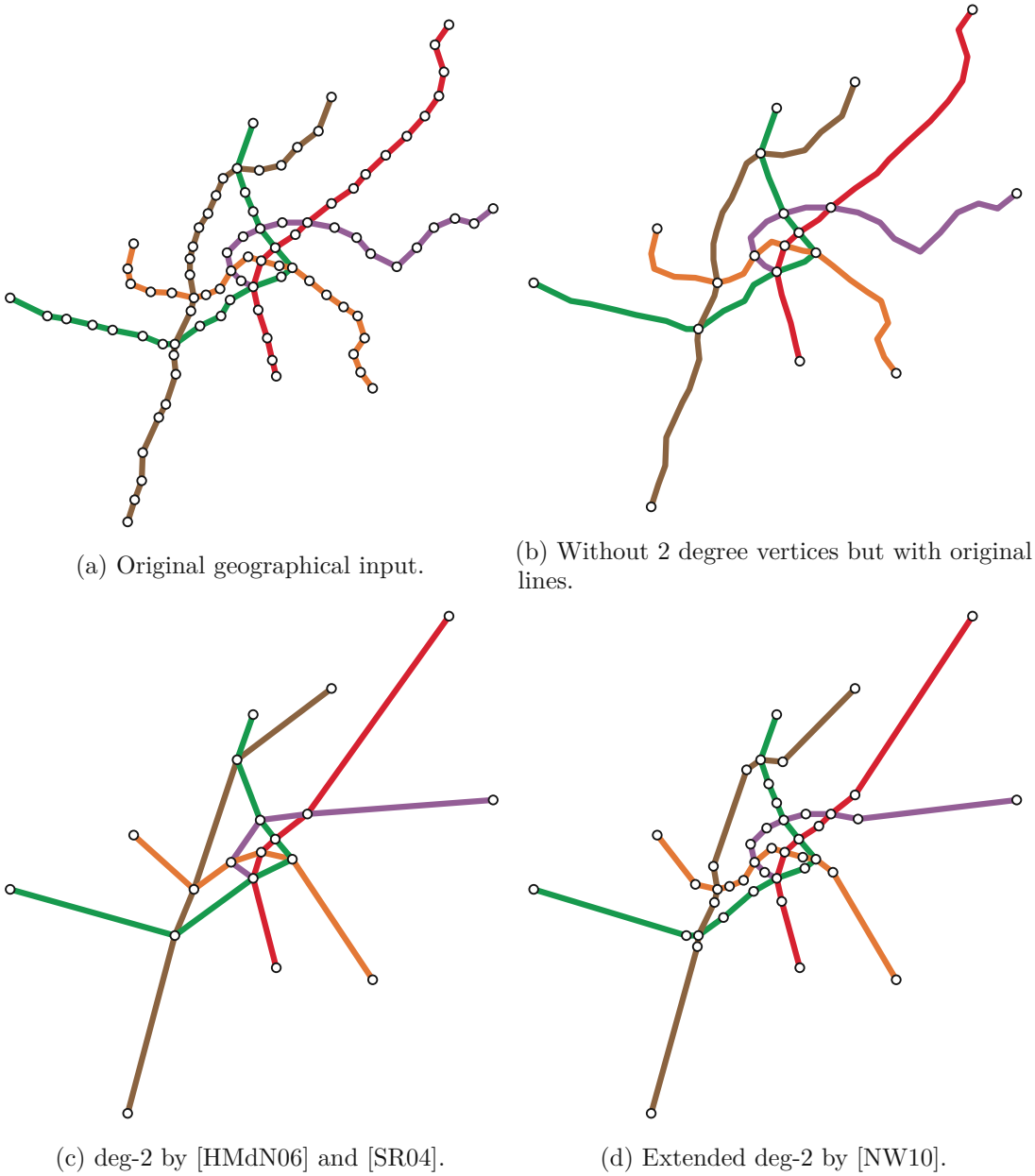
As proposed by [HMdN06] and [SR04] it is possible to reduce the size of the input graph of a metro network by deleting each vertex with degree 2 and connecting its neighbors with an edge. This procedure is a heuristic that is called two degree heuristic or only deg-2. In a more detailed formulation we look for a path $P_n = (v_1, \ldots, v_n)$ of any length n, with $deg(v_1) \neq 2$ and $deg(v_n) \neq 2$ but with $deg(v_i) = 2$ for each inner vertex $v_i : 1 < i < n$. We delete $v_2, \ldots, v_{n-1}$ and their adjacent edges and replace them with the edge (v_1, v_n). This edge is then drawn as a straight line in the output, because we produce straight line drawings. The deleted vertices are placed with equal distances along the edge (v_1, v_n) in the output drawing. To do so it is necessary to draw this edge with length at least $n + 1$, hence we set $L_{min}(v_1, v_n) = n + 1$.

The reason why this preprocessing step makes sense is that metro lines in most cases are paths. These paths are commonly connected into an overall network in a way that a lot of stations still have only two edges. Because of the design rule S1 the goal is to draw the lines that cross these station as straight lines. This means the subpaths out of vertices with $deg = 2$ should be drawn as straight line, hence they can be represented by one edge with respect to the summed up length. Note that it is not necessarily the case that in an optimal drawing all paths of degree 2 are drawn without bends, which makes this procedure a heuristic.

These properties can be observed on the metro map of Vienna as an example. The original layout with stations on geographical coordinates is given in Figure 2.10a. This can be compared to the same map after the described preprocessing step given in Figure 2.10c. The map given in Figure 2.10b is an intermediate step for illustration purposes. Deleting all vertices with $deg(v) = 2$ as done here shows the line bends that will get deleted after straightening the edges. This shows how sharp these deleted bends are. We can see that after the two degree heuristic is applied, the structure of the graph is still preserved, but it has a lot less vertices. This decreases the computational complexity.

One can imagine that there is some space for enhancement, because this kind of contraction of edges and vertices could lead to infeasible instances. That's because there is less latitude for edge directions to evade in dense parts of the map. This behavior tends to occur in city centers where a lot of lines are crossing at only a couple of stations. This problem can be prevented by leaving some of the vertices on each path, to allow a line bend at one of these vertices. In detail [NW10] proposed to leave the two outer vertices on each path that is connected to the rest of the graph on both endpoints. We call these paths internal paths, the endpoints x_1 and x_2 of internal paths have $deg(x_1) = deg(x_2) \geq 3$. On paths to a terminal station s that has $deg(s) = 1$ they propose to leave exactly one vertex. It should be the vertex furthest apart from the end station.

These two procedures and their differences are graphically shown in Figure 2.11. Figure 2.11a and Figure 2.11b both show input paths. Note that Figure 2.11a is an internal path, as x_1 and x_2 have $deg(x_1) = deg(x_2) = 4$. In contrast to that Figure 2.11b is a path to the terminal station x_2. It has $deg(x_2) = 1$. For x_1 only $deg(x_1) \neq 2$ must



(a) Original geographical input.

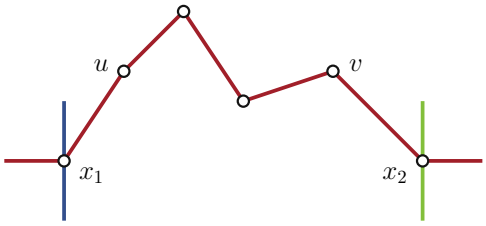
(b) Without 2 degree vertices but with original lines.

(c) deg-2 by [HMdN06] and [SR04].

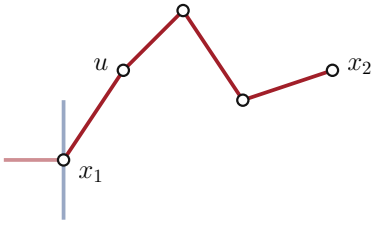
(d) Extended deg-2 by [NW10].

Figure 2.10: Metro map of Vienna during different preprocessing steps.

hold, but it is quite unusual that x_1 is a terminal station as well, because this would mean the whole graph only consists of this one path. Hence $deg(x_1) \geq 3$ holds in most cases and $deg(x_1) = 4$ holds in this example. In Figure 2.11c and Figure 2.11d the transformation done by [NW10] is shown. The neighbors of x_1 and x_2 stay in the graph on the internal path, these are u as neighbor of x_1 and v as neighbor of x_2. All other vertices in between are deleted. On the path to the terminal station x_2 only the neighbor

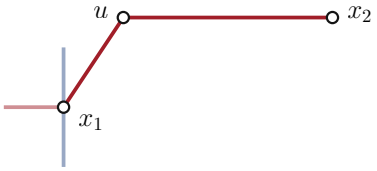


(a) Example of an internal path with endpoints x_1 and x_2, that are connected to other stations distinct to the stations of the path.



(b) Example of a path to a terminal station x_2. Note that x_1 could be connected to other stations or be a terminal station as well

(c) Extended deg-2 by [NW10] on internal paths.



(d) Extended deg-2 by [NW10] on paths to terminal stations.

(e) deg-2 by [HMdN06] and [SR04] on internal paths.

(f) deg-2 by [HMdN06] and [SR04] on paths to terminal stations.

Figure 2.11: Detailed explanation of the modifications done by the two degree heuristic as defined by [HMdN06] and [SR04] in comparison with [NW10] on paths with only inner vertices of degree 2. The difference between internal paths and paths to terminals is shown for the approach of [NW10]. There is no such difference in the approach of [HMdN06] and [SR04].

u of x_1 is kept, all other vertices between u and x_2 are deleted. In contrast to that the approach of [HMdN06] and [SR04] is shown in Figure 2.11e and Figure 2.11f. In both cases all vertices between x_1 and x_2 are deleted.

CHAPTER 3

SAT model

A boolean variable is a variable with a domain D of size $|D| = 2$. Commonly used are the domains $\{true, false\}$ or $\{1, 0\}$, hence the value assigned to a boolean variable is a truth value and can either have the meaning $true = 1$ or $false = 0$. A boolean formula F is a logical connection of boolean variables with logical operators. The basic operators are the binary operators conjunction $\wedge$ and disjunction $\vee$ and the unary operator negation $\neg$. There are other operators like an implication $\rightarrow$, that can be build by the basic operators and are not used in this paper because of that. A literal is either a variable a or a negated variable $\neg a$. Additionally instead of variables one can use the constants $\top = true$ and $\bot = false$ as well.

Boolean formulas can be inductively defined: Each literal is a formula. Constants $\top = true$ and $\bot = false$ are formulas as well. If G and H are formulas, then $(\neg G)$ (and $\neg H$ respectively), $(G \wedge H)$ and $(G \vee H)$ are also formulas. We also call G and H subformulas in this setting. Brackets can be omitted for better readability if they are not necessary, for example $((a \vee b) \vee c) = a \vee b \vee c$.

Formulas generated like this can have an arbitrary form as long as these rules are taken into account. A clause is a formula that is only a disjunction of literals. A formula F is in conjunctive normal form (CNF) [Pfa10] if it is a set of clauses, which are combined by conjunctions. Hence CNF formulas look as follows:

$$\bigwedge_{i=1}^{n}(\bigvee_{j=1}^{m} l_{ij}) = (l_{11} \vee \cdots \vee l_{1m}) \wedge \cdots \wedge (l_{n1} \vee \cdots \vee l_{nm}) \tag{1.1}$$

The conjunction of two subformulas $A \wedge B$ is interpreted as *true* only if both subformulas are interpreted as *true*. The disjunction of two subformulas $A \vee B$ is interpreted as *true* if at least one of the subformulas is interpreted as *true*. A variable is interpreted as its variable assignment.

A formula F is *satisfiable* if there exists a variable assignment such that F is interpreted as *true* under that assignment. An assignment with this characteristics is called a model of F. It is *unsatisfiable* if no such assignment exists. In the SAT problem we want to know if a given input formula F is *satisfiable* and if it is we sometimes want to obtain a model of F (but it is not necessary to ask for a model).

Problem 3 (SAT)**.**

Input:	*Boolean formula F.*
Question:	*Is F satisfiable?*
Output:	*Yes or no.*
(Optional)	*A variable assignment that satisfies F (if F is satisfiable).*

Conventional SAT solvers take input formulas only in CNF, hence we use a restricted form of the SAT problem that we call SAT-CNF.

Problem 4 (SAT-CNF)**.**

Input:	*Boolean formula F in conjunctive normal form.*
Question:	*Is F satisfiable?*
Output:	*Yes or no.*
(Optional)	*A variable assignment that satisfies F (if F is satisfiable).*

This restriction is no problem, because it is possible to translate every boolean formula into CNF.

Another variant of the SAT problem is the MaxSAT problem. The input of a MaxSAT instance is a formula in CNF given as a set of hard clauses and a set of weighted soft clauses. The goal is to find a variable assignment that satisfies all hard clauses and as many of the soft clauses as possible with respect to their weights.

Problem 5 (MaxSAT)**.**

Input:	*Boolean formula in conjunctive normal form as a set of hard clauses F_H and a set of (weighted) soft clauses F_S with a set of corresponding weights W.*
Question:	*Which model of F_H satisfies the most clauses in F_S (w.r.t. W)?*
Output:	*A variable assignment that satisfies F_H (if F_H is satisfiable) or UNSAT (if F_H is unsatisfiable)*

In this chapter we model constraints of the metro map problem as set of hard and soft clauses.

3.1 Unary encoding

Every integer is considered to be bound in this section and from now on we use $a, b, \ldots$ for integer variables, a^i for boolean variables, and $i, j, k, \ldots$ for (fixed) scalar values in this thesis. To encode integers in our SAT model, we use unary encoding as defined by

[YDG+16]. Each integer a that is bound by $l_a \leq a \leq u_a$ is encoded by $u_a - l_a$ different boolean variables $(a^{l_a+1}, a^{l_a+2}, \ldots, a^{u_a-1}, a^{u_a})$. Note that we omit the boolean variable a^{l_a} as $a \geq l_a$ has to be true in every assignment. The state $a^i = 1$ in unary encoding is equivalent to the constraint $a \geq i$, hence the fact $a = i$ is encoded by the assignment in Table 3.1.

a^{l_a+1}	a^{l_a+2}	$\cdots$	a^{i-1}	a^i	a^{i+1}	a^{i+2}	$\cdots$	a^{u_a-1}	a^{u_a}
1	1	$\cdots$	1	1	0	0	$\cdots$	0	0

Table 3.1: Integer $a = i$ in unary encoding

To ensure this behaviour, we add the following implication clauses for each integer variable a:

$$\neg a^i \vee a^{i-1} \qquad \forall i : l_a + 2 \leq i \leq u_a \tag{2.1}$$

With this we can easily model the constraint $a = i$ as the two clauses a^i and $\neg a^{i+1}$ (or just a^i if $i = u_a$). If $i < l_a$ or $i > u_a$, $a = i$ can never be true, hence we add the clause $\bot$.

We can also model inequalities between two integer variables a and b using several clauses, e.g. for $a \leq b$:

$$\neg a^i \vee b^i \qquad \forall i : \max\{l_a, l_b\} < i \leq \min\{u_a, u_b\} \tag{3.1}$$
$$\neg a^{u_b+1} \tag{3.2}$$
$$b^{l_a} \tag{3.3}$$

The main constraints in (3.1) are obtained by $a \geq i \rightarrow b \geq i$. By $l_a \leq a \leq b \leq u_b$ we get $l_a \leq b$ and $a \leq u_b$, hence the clauses (3.2) and (3.3) have to be added. Note that there is only a linear number of constraints (in the overlapping range of the variables) instantiated, in contrast to the quadratic number of constraints necessary in a naive approach

Offsets as in $a + g \leq b$ with g as constant can easily be modelled by adding the offset g to the index of b while paying attention to the range of b.

$$\neg a^i \vee b^{i+g} \qquad \forall i : \max\{l_a, l_b - g\} < i \leq \min\{u_a, u_b - g\} \tag{4.1}$$
$$\neg a^{u_b-g+1} \tag{4.2}$$
$$b^{l_a+g} \tag{4.3}$$

We also use the fact that $a = b$ is equivalent to the two inequalities $a \leq b$ and $a \geq b$ to

model the equality $a = b$

$$\begin{aligned} &\neg a^i \vee b^i \\ &\quad a^i \vee \neg b^i \end{aligned} \qquad \forall i : \max\{l_a, l_b\} < i \leq \min\{u_a, u_b\} \tag{5.1}$$

$$\begin{aligned} &\neg a^{u_b+1} \\ &\neg b^{u_a+1} \\ &\quad a^{l_b} \\ &\quad b^{l_a} \end{aligned} \tag{5.2}$$

Coefficients can be used similarly as we did with offsets to model (in)equalities like $a = b \cdot g$.

$$\neg a^i \vee b^{\lceil \frac{i}{g} \rceil} \qquad \forall i : \max\{l_a, l_b \cdot g\} < i \leq \min\{u_a, u_b \cdot g\} \tag{6.1}$$

$$a^{j \cdot g} \vee \neg b^j \qquad \forall j : \max\{\frac{l_a}{g}, l_b\} < j \leq \min\{\frac{u_a}{g}, u_b\} \tag{6.2}$$

$$\neg a^{u_b \cdot g+1} \tag{6.3}$$

$$\neg b^{\lfloor \frac{u_a}{g} \rfloor+1} \tag{6.4}$$

$$a^{l_b \cdot g} \tag{6.5}$$

$$b^{\lceil \frac{l_a}{g} \rceil} \tag{6.6}$$

Inequalities containing three different integer variables are more complex. With two variables we needed a linear number of constraints in the variable range, but now we need a quadratic number to constrain each combination of two indices. For $a - b \leq c$ this leads to the following clauses:

$$\neg a^i \vee b^j \vee c^{i-j+1} \qquad \begin{array}{l} \forall i : l_a < i \leq u_a \\ \forall j : l_b < j \leq u_b \end{array} \text{ if } l_c < i - j + 1 \leq u_c \tag{7.1}$$

$$\neg a^{u_b+u_c+1} \tag{7.2}$$

$$b^{l_a-u_c} \tag{7.3}$$

$$c^{l_a-u_b} \tag{7.4}$$

$$\neg a^i \vee c^{i-u_b} \qquad \forall i : l_a < i \leq u_a \text{ if } l_c < i - u_b \leq u_c \tag{7.5}$$

$$b^i \vee c^{l_a-i} \qquad \forall i : l_b < i \leq u_b \text{ if } l_c < l_a - j \leq u_c \tag{7.6}$$

$$\neg a^i \vee b^{i-u_c} \qquad \forall i : l_a < i \leq u_a \text{ if } l_b < i - u_c \leq u_b \tag{7.7}$$

Again as before we can restrict the values for each integer variable using the bounds of the other variables as in (7.2)-(7.4). From $a \leq c + b \leq u_b + u_c$ we get $\neg a^{u_b+u_c+1}$, from $l_a - u_b \leq a - b \leq c$ we get $c^{l_a-u_b}$ and $b^{l_a-u_c}$ respectively.

Additionally we forbid each combination of two variables that would exceed the range of the third variable as shown in (7.5), (7.6) and (7.7).

The constraints for $a + b + g = c$ are a combination of all rules used before. The equality is implemented by two inequalities, while offsets occurring on one side are implemented as before by adding them to the index of variables on the other side of the equality.

$$\neg a^i \vee \neg b^j \vee c^{i+j+g} \qquad \forall i : l_a < i \leq u_a \; \forall j : l_b < j \leq u_b \text{ if } l_c < i + j + g \leq u_c \tag{8.1}$$

$$a^i \vee b^j \vee \neg c^{i+j-1+g} \qquad \forall i : l_a < i \leq u_a \; \forall j : l_b < j \leq u_b \text{ if } l_c < i + j - 1 + g \leq u_c \tag{8.2}$$

$$\neg a^{u_c - l_b - g + 1} \tag{8.3}$$

$$\neg b^{u_c - l_a - g + 1} \tag{8.4}$$

$$c^{l_a + l_b + g} \tag{8.5}$$

$$a^{l_c - u_b - g} \tag{8.6}$$

$$b^{l_c - u_a - g} \tag{8.7}$$

$$\neg c^{u_a + u_b + g + 1} \tag{8.8}$$

$$a^i \vee b^{l_c - i - g + 1} \qquad \forall i : l_a < i \leq u_a \text{ if } l_b < l_c - i - g + 1 \leq u_b \tag{8.9}$$

$$\neg a^i \vee \neg b^{u_c - i - g + 1} \qquad \forall i : l_a < i \leq u_a \text{ if } l_b < u_c - i - g + 1 \leq u_b \tag{8.10}$$

$$\neg a^i \vee c^{l_b + i + g} \qquad \forall i : l_a < i \leq u_a \text{ if } l_c < l_b + i + g \leq u_c \tag{8.11}$$

$$\neg b^i \vee c^{l_a + i + g} \qquad \forall i : l_b < i \leq u_b \text{ if } l_c < l_a + i + g \leq u_c \tag{8.12}$$

$$a^i \vee \neg c^{u_b + i + g} \qquad \forall i : l_a < i \leq u_a \text{ if } l_c < u_b + i + g \leq u_c \tag{8.13}$$

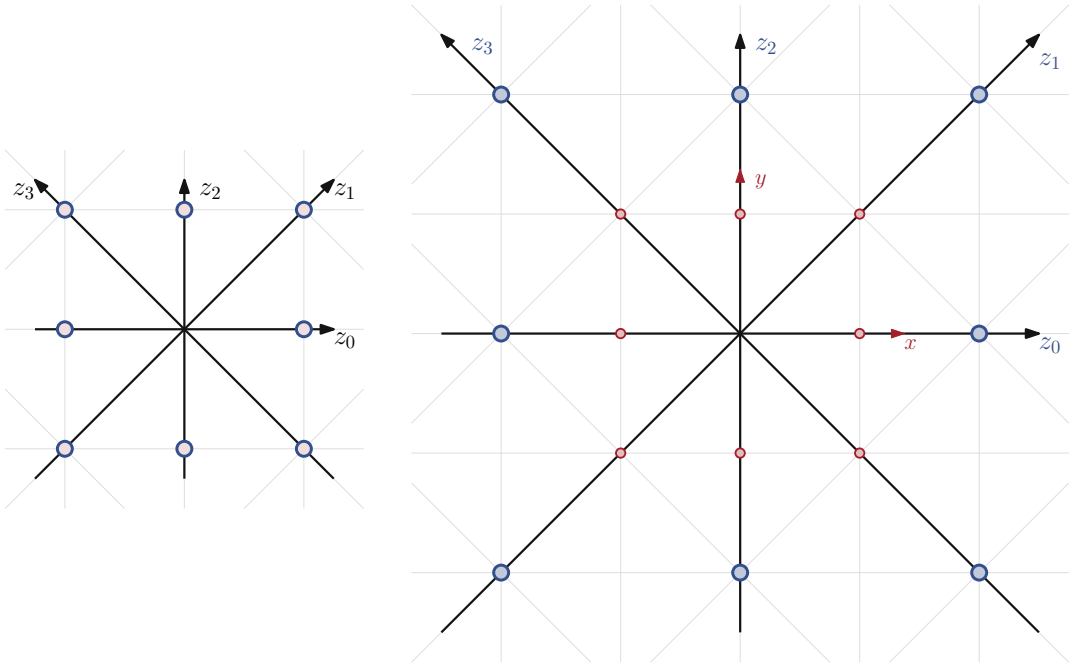
$$b^i \vee \neg c^{u_a + i + g} \qquad \forall i : l_b < i \leq u_b \text{ if } l_c < u_a + i + g \leq u_c \tag{8.14}$$

We still have to disallow the combinations that would outrange the boundaries of the variables in (8.3) to (8.14).

If we have a set of clauses $[c_1, \ldots, c_n]$ and want the disjunction $l_1 \vee \cdots \vee l_m \vee (c_1 \wedge \cdots \wedge c_n)$ to be added, we use De Morgan's laws [DM47] to generate the clauses (9.1).

$$l_1 \vee \cdots \vee l_m \vee c_i \qquad \forall i : 1 \leq i \leq n \tag{9.1}$$

Note that in the implementation we check whether the boundaries of the integer variables allow to satisfy the respective inequalities at all beforehand. If not we add a set of clauses that is impossible to satisfy. These are the two clauses a^i and $\neg a^i$ together for some integer variable a and some index $l_a < i \leq u_a$. In general we add these clauses whenever we have to add $\bot$, for example when we would have to add $a = i$ for some $l_a \not\leq i$ or $i \not\leq u_a$. This can make the generated formula F unsatisfiable, hence the map gets impossible to draw, if a^i and $\neg a^i$ are added as clauses to F as they are. In most cases this will not happen. The constraints added in the following sections are not an (in)equality itself as clauses. The constraints are disjunctions of some literals $l_1, \ldots, l_m$ and the (in)equality. For unsatisfiable (in)equalities this leads to the clause $l_1 \vee \cdots \vee l_m \vee \bot$ that is added as



(a) L^∞ metric [NW10] (b) Modified L^∞ metric

Figure 3.1: Octolinear coordinate system using different versions of the L^∞ metric.

the two clauses $l_1 \vee \cdots \vee l_m \vee a^i$ and $l_1 \vee \cdots \vee l_m \vee \neg a^i$ using the rule described in (9.1). These clauses are still satisfiable by assigning *true* to one of the literals $l_1, \ldots, l_m$.

In the following sections we use integer variables and (in)equalities for better readability. In the implementation these are replaced by clauses as defined in this section.

3.2 Coordinates

In a schematic metro map each vertex $v \in V$ of the input graph G has $x(v)$ and $y(v)$ coordinates in the plane. In an optimized drawing we can use an underlying grid and represent these coordinates with integer variables. To use unary encoding for integer variables it is necessary that the values of these variables are bounded. To do so we introduce additional problem input values x_{max} and y_{max} to restrict the drawing to a grid of size $x_{max} \times y_{max}$ with the following variable bounds:

$$0 \leq x(v) \leq x_{max} \qquad \forall v \in V$$
$$0 \leq y(v) \leq y_{max} \qquad \forall v \in V$$

Note that in our implementation it is possible to bound $y(v)$ by some value $y_{max} \neq x_{max}$, but for the experiments we used $y_{max} = x_{max}$.

In our case the drawing should be octolinear, hence we have four directions ($d \in [0, 1, 2, 3]$). To model all needed constraints we create four additional integer variables representing the Cartesian coordinates ($x(v)\widehat{=}z_0(v), y(v)\widehat{=}z_2(v), z_1(v), z_3(v)$) using the infinity metric from [NW10] as shown in Figure 3.1a with one modification shown in Figure 3.1b. In the original metric, $z_0(v) = x(v)$ and $z_2(v) = y(v)$ holds for each vertex $v \in V$ and so does $z_1(v) = (z_0(v) + z_2(v))/2$ and $z_3(v) = (z_2(v) - z_0(v))/2$. With this the grid points marked in Figure 3.1a have the same uniform distance from the origin, hence the distance to the origin is 1. But the coordinates do not have integer values in all Cartesian coordinates. The right point in the middle for example has Cartesian coordinates ($z_0 = 1, z_2 = 0, z_1 = 0.5, z_3 = 0.5$). With our simple implementation of unary encoding we can only model integer values, so it is necessary to scale the coordinate grid as shown in Figure 3.1b. This is done by multiplying the z_0 and z_2 values by 2 and thereby also z_1 and z_3 by definition, to ensure that all of them are integers.

This leads to the constraints (10.1) to (10.4).

$$z_0(v) = 2x(v) \qquad \forall v \in V \tag{10.1}$$
$$z_1(v) = x(v) + y(v) \qquad \forall v \in V \tag{10.2}$$
$$z_2(v) = 2y(v) \qquad \forall v \in V \tag{10.3}$$
$$z_3(v) = y(v) - x(v) \qquad \forall v \in V \tag{10.4}$$

Using four Cartesian coordinates is done for notational convenience and not strictly necessary. It would be sufficient to use two linearly independent coordinates instead.

Using the bounds for $x(v)$ and $y(v)$ the constraints lead to bounds for these integer variables as well:

$$0 \leq z_0(v) \leq 2x_{max} \qquad \forall v \in V$$
$$0 \leq z_1(v) \leq x_{max} + y_{max} \qquad \forall v \in V$$
$$0 \leq z_2(v) \leq 2y_{max} \qquad \forall v \in V$$
$$-x_{max} \leq z_3(v) \leq y_{max} \qquad \forall v \in V$$

When working with some coordinate $z_i(v)$ in our model, we use the coordinate in orthogonal direction $z_i^o(v) = z_{i+2 \mod 4}(v)$ in some cases. We use this as an abbreviation in the notation, and the implementation simply uses the appropriate z_i variable. Note that while there are no z_i^o variables in the implementation, the z_1 and z_3 variables actually exist in the implementation, even if they would not be not strictly necessary as mentioned before.

Additionally, we introduce direction values for each edge $(u, v) \in E$ and its counter (v, u), modelled by integer variables $0 \leq dir(u, v) < 8$ and $0 \leq dir(v, u) < 8$. These values depend on the coordinates of the adjacent vertices $z_0(u)$, $z_1(u)$, $z_2(u)$, $z_3(u)$ and $z_0(v)$, $z_1(v)$, $z_2(v)$, $z_3(v)$. In fact the values of $dir(u, v)$ are further restricted to the values in $S(u, v)$. The constraints in Subsection 3.3.1 ensure that $dir(u, v) = dir(v, u) + 4 \mod 8$ holds.

3.3 Hard constraints

The hard constraints of the SAT model ensure four things: the minimum edge length, the direction of edges to generate an octolinear drawing, the preservation of the combinatorial embedding and planarity. These are the constraints defined in Chapter 2 that a schematic metro map drawing has to satisfy.

3.3.1 Edge directions and minimum length

For each edge $(u, v) \in E$ we introduce a set of boolean variables $\alpha_i(u, v)$ for each $i \in S(u, v)$. Exactly one of these variables is *true* for each edge, this is enforced by constraints (11.1) and (11.2). The variable $\alpha_i(u, v)$ is true if and only if the edge (u, v) has direction i. This is modelled in constraints (11.3) to (11.4). It also ensures that the directions of (u, v) and (v, u) match up.

$\forall (u, v) \in E$:

$$\bigvee_{i \in S(u,v)} \alpha_i(u, v) \tag{11.1}$$

$$\neg\alpha_i(u, v) \vee \neg\alpha_j(u, v) \qquad \forall i, j \in S(u, v) : i < j \tag{11.2}$$

$$\neg\alpha_i(u, v) \vee dir(u, v) = i \qquad \forall i \in S(u, v) \tag{11.3}$$

$$\neg\alpha_i(u, v) \vee dir(v, u) = (i + 4) \mod 8 \qquad \forall i \in S(u, v) \tag{11.4}$$

$$\neg\alpha_i(u, v) \vee z_i^o(u) = z_i^o(v) \qquad \forall i \in S(u, v) \tag{11.5}$$

$$\neg\alpha_i(u, v) \vee z_i(u) + 2L_{min}(u, v) \leq z_i(v) \qquad \forall i \in S(u, v) : i < 4 \tag{11.6}$$

$$\neg\alpha_i(u, v) \vee z_{i-4}(v) + 2L_{min}(u, v) \leq z_{i-4}(u) \qquad \forall i \in S(u, v) : i \geq 4 \tag{11.7}$$

Note that we defined $z_i^o(u)$ in (11.5) as the variables in orthogonal direction of the variables $z_i(v)$. If an edge (u, v) has some direction i, then the vertices u and v have the same coordinates in the orthogonal direction $i + 2 \mod 4$ as shown in Figure 3.2.

In (11.6) to (11.7) the minimal length $L_{min}(u, v)$ of the edge (u, v) is ensured by forcing the distance between u and v to be at least $L_{min}(u, v)$ in the edges direction.

3.3.2 Combinatorial embedding / Circular vertex orders

For each vertex $v \in V$ with more than two neighbors we introduce a set of boolean variables $(\beta_1(v), \ldots, \beta_{deg(v)}(v))$ containing one variable for each neighbor of v. Note that $deg(v)$ is defined as the number of neighbors of v. We also define $(u_1, \ldots, u_{deg(v)})$ as the circular ordered neighbors of vertex v. To keep the combinatorial order of the graph, we preserve the circular order of the neighboring vertices of v. We omit the vertices with less than three neighbors, because in any combination of edge directions for vertices with fewer neighbors the circular order trivially is preserved. To do so for vertices v with $deg(v) > 2$, we ensure that the direction values of the edges (v, u_i) to the neighbors are strictly increasing when visiting in the given input order. There is one exception when

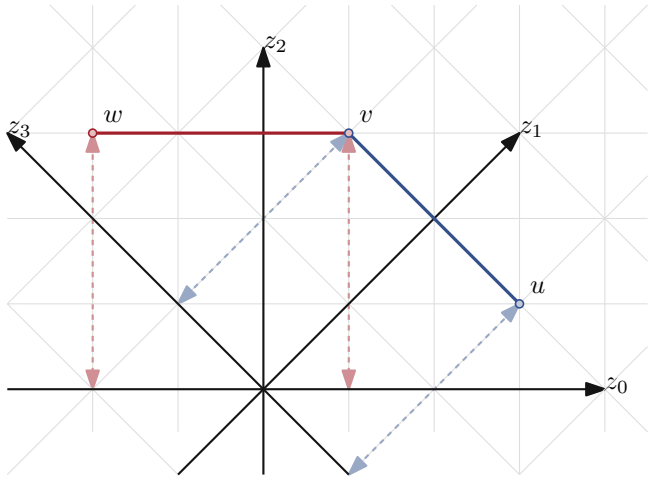


Figure 3.2: Edge directions. The edge (u, v) (in blue) is drawn in direction 3, hence it is drawn parallel to the z_3 axis. That is the reason why its distance to the origin in the orthogonal direction $1 = 3 + 2 \mod 4$ is the same for each point of the edge This distance is indicated by the blue arrows. The same holds for edge (v, w) (in red).

we reach the last sector and switch to the first sector again. Exactly one β_i variable has to be true modeling this exception.

$\forall v \in V : deg(v) > 2 :$

$$\bigvee_{i=1}^{deg(v)} \beta_i(v) \tag{12.1}$$

$$\neg\beta_i(v) \vee \neg\beta_j(v) \qquad \forall i \in \mathbb{N} : 1 \leq i < j \leq deg(v) \tag{12.2}$$

$$\beta_i(v) \vee dir(v, u_i) + 1 \leq dir(v, u_{i+1}) \qquad \forall i \in \mathbb{N} : 1 \leq i < deg(v) \tag{12.3}$$

$$\beta_i(v) \vee dir(v, u_{deg(v)}) + 1 \leq dir(v, u_1) \tag{12.4}$$

$\forall v \in V : deg(v) = 2 :$

$$\neg\alpha_i(v, u_1) \vee \neg\alpha_i(v, u_2) \qquad \forall i \in S(v, u_1) : i \in S(v, u_2) \tag{13.1}$$

3.3.3 Planarity / Edge spacing

To guarantee planarity, we must disallow edge crossing for each pair of not connected edges. Two non-crossing edges can be separated by one straight line. In fact non-crossing edges in octolinear drawings can be separated by an octolinear line. We define E' as the set of edge pairs that are not adjacent. For each pair of edges $(e, e') \in E'$ we introduce a set of boolean variables $\gamma_i(e, e')$ with $0 \leq i < 8$. Each variable corresponds to one possible direction. As the two edges have to be separated by a line in at least one of the directions, at least one of the variables in each set have to be true.

$$\bigvee_{i=0}^{7} \gamma_i(e, e') \qquad \forall (e, e') \in E' \tag{14.1}$$

If, for example, the edges $e = (u, v)$ and $e' = (u', v')$ are separated by a horizontal line, then the coordinate values in vertical direction, hence the z_2 values, of the vertices u and v have to be strictly smaller than t he z_2 values of the vertices u' and v'. More precisely their values have to differ by at least d_{min} (the minimal distance between edges).

$\forall (e = (u, v), e' = (u', v')) \in E', \forall i \in \mathbb{N} : 0 \leq i < 4 :$

$$\neg\gamma_i(e, e') \vee z_i(u) + d_{min} \leq z_i(u') \tag{15.1}$$
$$\neg\gamma_i(e, e') \vee z_i(v) + d_{min} \leq z_i(u') \tag{15.2}$$
$$\neg\gamma_i(e, e') \vee z_i(u) + d_{min} \leq z_i(v') \tag{15.3}$$
$$\neg\gamma_i(e, e') \vee z_i(v) + d_{min} \leq z_i(v') \tag{15.4}$$

This constraints consider the case that e has the smaller values in the corresponding direction. In fact also e' could have the smaller values which is modelled in the second batch of constraints:

$\forall (e = (u, v), e' = (u', v')) \in E', \forall i \in \mathbb{N} : 0 \leq i < 4 :$

$$\neg\gamma_{i+4}(e, e') \vee z_i(u') + d_{min} \leq z_i(u) \tag{16.1}$$
$$\neg\gamma_{i+4}(e, e') \vee z_i(u') + d_{min} \leq z_i(v) \tag{16.2}$$
$$\neg\gamma_{i+4}(e, e') \vee z_i(v') + d_{min} \leq z_i(u) \tag{16.3}$$
$$\neg\gamma_{i+4}(e, e') \vee z_i(v') + d_{min} \leq z_i(v) \tag{16.4}$$

For better efficiency, only pairs of edges on the same face have to be checked, hence the set E' will be smaller.

$$E' = \{(e, e') \mid e, e' \in E : e \cap e' = \emptyset \wedge e \text{ and } e' \text{ are on the same face}\} \tag{17.1}$$

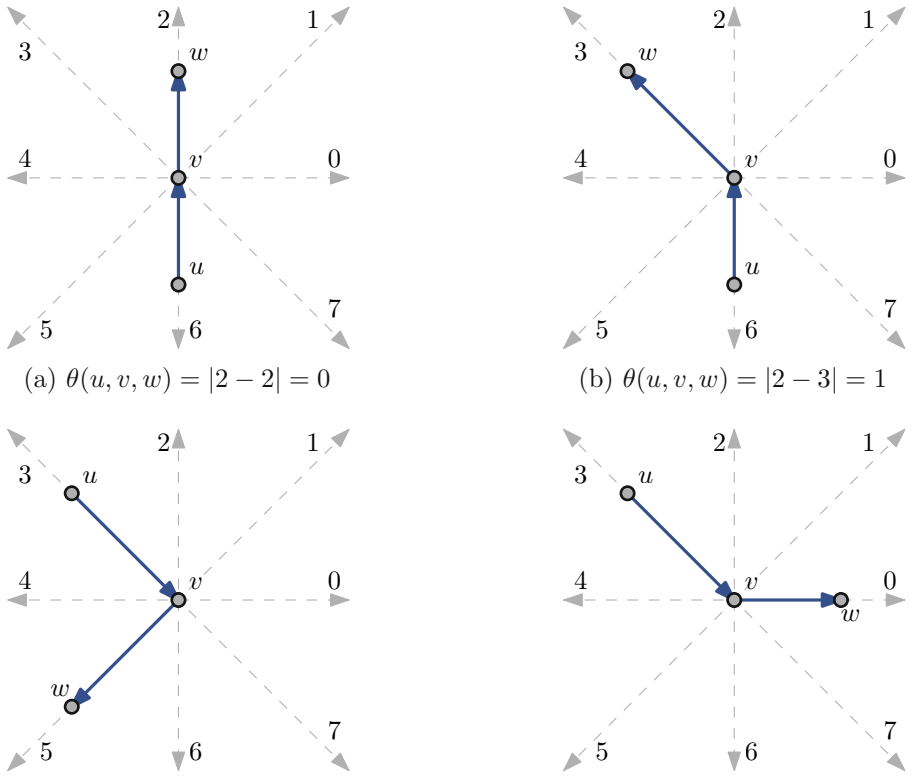
That's because if two edges e_1 and e_2 of different faces cross, there have to be two edges on the face of e_1 or on the face of e_2 that cross as well. Nevertheless the planarity constraints are still the most constraints in big O notation. This is the reason why Nöllenburg and Wolff [NW10] suggested to add these constraints only on demand, hence we calculate solutions without these constraints, check for crossings and if we find some we only add the corresponding constraints before recalculating a solution.

3.4 Optimization

To optimize (minimize) the number of line bends, the relative position of edges and the compactness of the drawing, some more hard constraints are needed. These are necessary to measure the values that have to be optimized.

3.4.1 Line bends

To measure line straightness we introduce an integer variable $\theta(u, v, w)$ for each pair of consecutive edges (u, v) and (v, w) on some line $L \in \mathcal{L}$. We calculate the gap between the directions $dir(u, v)$ and $dir(v, w)$ and store it in $\theta(u, v, w)$. This gives the bounds $0 \leq \theta(u, v, w) < 4$. By [NW10] we know that $\theta(u, v, w) = \min\{|dir(u, v) - dir(v, w)|, 8 - |dir(u, v) - dir(v, w)|\}$ holds as illustrated in Figure 3.3.



(a) $\theta(u, v, w) = |2 - 2| = 0$ (b) $\theta(u, v, w) = |2 - 3| = 1$

(c) $\theta(u, v, w) = |7 - 5| = 2$ (d) $\theta(u, v, w) = 8 - |7 - 0| = 1$

Figure 3.3: Calculation of $\theta(u, v, w)$ for different expressions of line bends.

To ensure these dependencies we introduce two binary correction variables $\delta_1(u, v, w)$ and $\delta_2(u, v, w)$ for each of these edge pairs. If $\theta(u, v, w) = |dir(u, v) - dir(v, w)|$ holds, then either the inequality in (18.1) or in (18.2) is true. In the first case $\theta(u, v, w) = dir(v, w) - dir(u, v)$ holds, hence the inequality in (18.4) is false, but the constraint can be true with $\neg\delta_2$. In the second case the same holds for (18.3) and $\neg\delta_1$ simultaneously. For $\theta(u, v, w) = 8 - |dir(u, v) - dir(v, w)| < i$ either the inequality in (18.3) or in (18.4) holds respectively. In general for (18.1) and (18.2) exactly one inequality is true as well as for (18.3) and (18.4). The other two constraints are satisfied via the binary correction variables.

$\forall L \in \mathcal{L},\ \forall (u,v),(v,w) \in L :$

$$\delta_2(u,v,w) \vee dir(u,v) + \theta(u,v,w) \geq dir(v,w) \tag{18.1}$$
$$\delta_1(u,v,w) \vee dir(v,w) + \theta(u,v,w) \geq dir(u,v) \tag{18.2}$$
$$\neg\delta_1(u,v,w) \vee dir(u,v) + \theta(u,v,w) - 8 \geq dir(v,w) \tag{18.3}$$
$$\neg\delta_2(u,v,w) \vee dir(v,w) + \theta(u,v,w) - 8 \geq dir(u,v) \tag{18.4}$$

To omit inequalities with at least three variables or summands, we translated these inequalities into (19.1) - (19.4).

$\forall L \in \mathcal{L},\ \forall (u,v),(v,w) \in L,\ \forall i \in \mathbb{N} : 1 \leq i < 4 :$

$$\delta_2(u,v,w) \vee \theta(u,v,w)^i \vee dir(u,v) + i > dir(v,w) \tag{19.1}$$
$$\delta_1(u,v,w) \vee \theta(u,v,w)^i \vee dir(v,w) + i > dir(u,v) \tag{19.2}$$
$$\neg\delta_1(u,v,w) \vee \theta(u,v,w)^i \vee dir(u,v) + i - 8 > dir(v,w) \tag{19.3}$$
$$\neg\delta_2(u,v,w) \vee \theta(u,v,w)^i \vee dir(v,w) + i - 8 > dir(u,v) \tag{19.4}$$

Note again that for any $1 \leq i < 4$ and any $0 \leq dir(u,v), dir(v,w)$ the inequality of exactly two of these four constraints is satisfied independent of the values $\delta_1(u,v,w)$, $\delta_2(u,v,w)$ and $\theta(u,v,w)^i$, if and only if $|dir(u,v) - dir(v,w)| < i$ or $8 - |dir(u,v) - dir(v,w)| < i$ holds. Then the variables $\delta_1(u,v,w)$ and $\delta_2(u,v,w)$ can be used to satisfy the other two constraints. Otherwise $\theta(u,v,w)^i$ has to be set to *true*, setting $\theta(u,v,w) = \min\{|dir(u,v) - dir(v,w)|, 8 - |dir(u,v) - dir(v,w)|\}$, what we wanted to achieve.

3.4.2 Relative positions

Similar to the constraints for line bends we introduce an integer variable $\xi(u,v)$ for each edge (u,v), that measures the gap between the direction $dir(u,v)$ in the drawing and the original section of the edge $sec_u(v)$. This gives the bounds $0 \leq \xi(u,v) \leq dev$, since dev is the maximal gap between these two values (see Section 2.2). We also introduce two binary correction variables $\eta_1(u,v)$ and $\eta_2(u,v)$ for each edge. $\xi(u,v,w) = \min\{|dir(u,v) - sec_u(v)|, 8 - |dir(u,v) - sec_u(v)|\}$ holds respectively.

$\forall (u,v) \in E :$

$$\eta_2(u,v) \vee sec_u(v) - dir(u,v) \leq \xi(u,v) \tag{20.1}$$
$$\eta_1(u,v) \vee dir(u,v) - sec_u(v) \leq \xi(u,v) \tag{20.2}$$
$$\neg\eta_1(u,v) \vee sec_u(v) - dir(u,v) + 8 \leq \xi(u,v) \tag{20.3}$$
$$\neg\eta_2(u,v) \vee dir(u,v) - sec_u(v) + 8 \leq \xi(u,v) \tag{20.4}$$

The constraints given in (20.1)-(20.4) are the equivalent to the first version of the line bend constraints given in (18.1)-(18.4). We use the same transformation to omit inequalities with three summands to get (21.1)-(21.4)

$\forall (u,v) \in E,\ \forall i \in \mathbb{N} : 1 \leq i \leq dev :$

$$\eta_2(u,v) \vee \xi(u,v)^i \vee sec_u(v) - dir(u,v) < i \tag{21.1}$$
$$\eta_1(u,v) \vee \xi(u,v)^i \vee dir(u,v) - sec_u(v) < i \tag{21.2}$$
$$\neg\eta_1(u,v) \vee \xi(u,v)^i \vee sec_u(v) - dir(u,v) + 8 < i \tag{21.3}$$
$$\neg\eta_2(u,v) \vee \xi(u,v)^i \vee dir(u,v) - sec_u(v) + 8 < i \tag{21.4}$$

There is one significant difference to the line bends constraints: the section $sec_u(v)$ is no variable but a constant, hence we can use the fact that $dir(u,v) > c$ can be modeled as one literal $dir(u,v)^{c+1}$ as well as $dir(u,v) < c$ can be modeled as $\neg dir(u,v)^c$ for some constant c. Using these facts the constraints can be modeled as:

$\forall (u,v) \in E,\ \forall i \in \mathbb{N} : 1 \leq i \leq dev :$

$$\eta_2(u,v) \vee \xi(u,v)^i \vee dir(u,v)^{sec_u(v)-i+1} \tag{22.1}$$
$$\eta_1(u,v) \vee \xi(u,v)^i \vee \neg dir(u,v)^{sec_u(v)+i} \tag{22.2}$$
$$\neg\eta_1(u,v) \vee \xi(u,v)^i \vee dir(u,v)^{sec_u(v)-i+9} \tag{22.3}$$
$$\neg\eta_2(u,v) \vee \xi(u,v)^i \vee \neg dir(u,v)^{sec_u(v)+i-8} \tag{22.4}$$

Again exactly two of the inequalities of the four constraints are satisfied, if and only if $|dir(u,v) - sec_u(v)| < i$ or $8 - |dir(u,v) - sec_u(v)| < i$ holds. Hence we can satisfy the other two constraints using the boolean variables $\eta_1(u,v)$ and $\eta_2(u,v)$. Otherwise $\xi(u,v)^i$ has to be *true* to achieve $\xi(u,v,w) = \min\{|dir(u,v) - sec_u(v)|, 8 - |dir(u,v) - sec_u(v)|\}$.

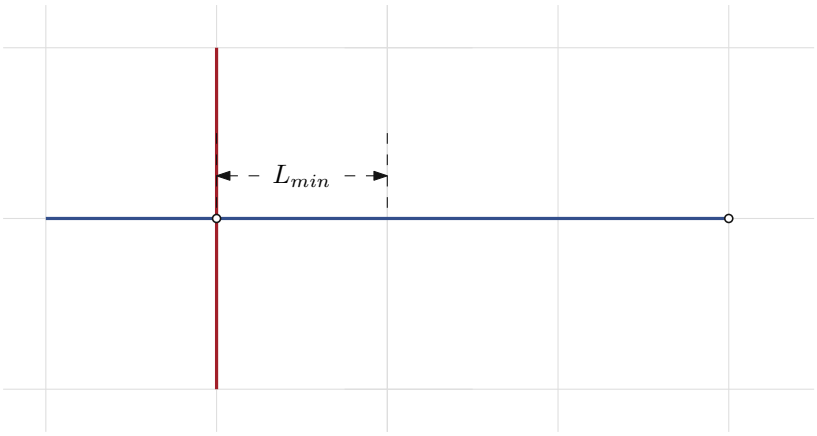
3.4.3 Compactness

Using the properties of the L^∞-metric, the length of an edge (u,v) is the maximum distance of u and v on the x-axis and the y-axis. We introduce an integer variable $\lambda(u,v)$ for each edge (u,v), that has to be at least as high as the distance in each of these two directions. To model this we add the clauses in (23.1) to (23.4).

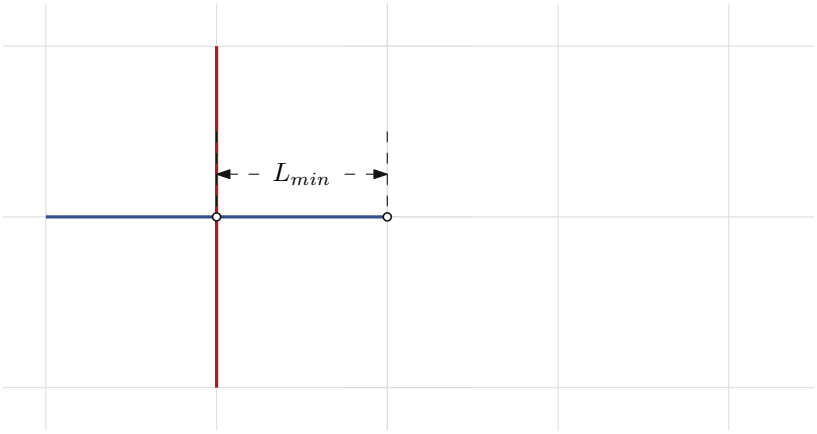
$$x(u) - x(v) \leq \lambda(u,v) \qquad \forall (u,v) \in E \tag{23.1}$$
$$x(v) - x(u) \leq \lambda(u,v) \qquad \forall (u,v) \in E \tag{23.2}$$
$$y(u) - y(v) \leq \lambda(u,v) \qquad \forall (u,v) \in E \tag{23.3}$$
$$y(v) - y(u) \leq \lambda(u,v) \qquad \forall (u,v) \in E \tag{23.4}$$

This works because with the L^∞-metric an edge in diagonal direction with length l has also length l on the x-axis as well as on the y-axis.

To gain more efficiency, we can force edges to end stations to have fixed lengths, because in an optimized drawing these edges have minimal length as illustrated in Figure 3.4. That's because if a drawing exists with some longer edge to an end station, this edge can



(a) Edge to the terminal station is longer than L_{min}



(b) Edge to the terminal station has length L_{min}

Figure 3.4: Edges to terminal stations have length L_{min} in optimized drawings. Figure 3.4a cannot be an optimized drawing because Figure 3.4b has less cost. That's because the cost for line bends is the same in both drawings as well as the cost for relative positions. The only difference is the length of the edge to the terminal station, hence the cost for edge lengths is smaller. Additionally if some hard constraint would be violated in the second drawing, then it has to be violated in the fist one too, so it also could not be an optimized drawing.

be shortened to minimal length and the new drawing would have smaller cost while still satisfying the same hard constraints, hence the original drawing was not optimal. To do so we have to add the hard clauses (24.1) instead for these edges.

$$\lambda(u,v) = L_{min}(u,v) \qquad \forall (u,v) \in E : deg(u) = 1 \vee deg(v) = 1 \tag{24.1}$$

These constraints can be translated to (25.1) and (25.2)

$$\lambda(u,v)^{L_{min}(u,v)} \qquad \forall(u,v) \in E : deg(u) = 1 \vee deg(v) = 1 \tag{25.1}$$

$$\neg\lambda(u,v)^{L_{min}(u,v)+1} \qquad \forall(u,v) \in E : deg(u) = 1 \vee deg(v) = 1 \tag{25.2}$$

3.4.4 Soft constraints

The objective function would now be calculated by summing up all θ, ξ and λ variables. To minimize this objective function we have to add the following soft constraints for each of these integer variables: The value has to be smaller than i for each $l < i \leq u$ with l being the lower bound and u the upper bound of the variable. Using unary encoding for integer variables has the benefit that we can use the clause $\neg a^i$ to encode the fact that a should be smaller than i.

Line Bends We add the soft clauses for line straightness with weight f_1:

$$\neg\theta(u,v,w)^i \qquad \forall L \in \mathcal{L},\ \forall(u,v),(v,w) \in L,\ \forall 1 \leq i < 4 \tag{26.1}$$

For user friendly readability of schematic metro maps it can be more important to have line straightness in stations with more then two adjacent edges than in the other stations (see Figure 3.5). Because of this in our implementation we have the opportunity to add these constraints for vertices v with $deg(v) > 2$ with weight $2f_1$.

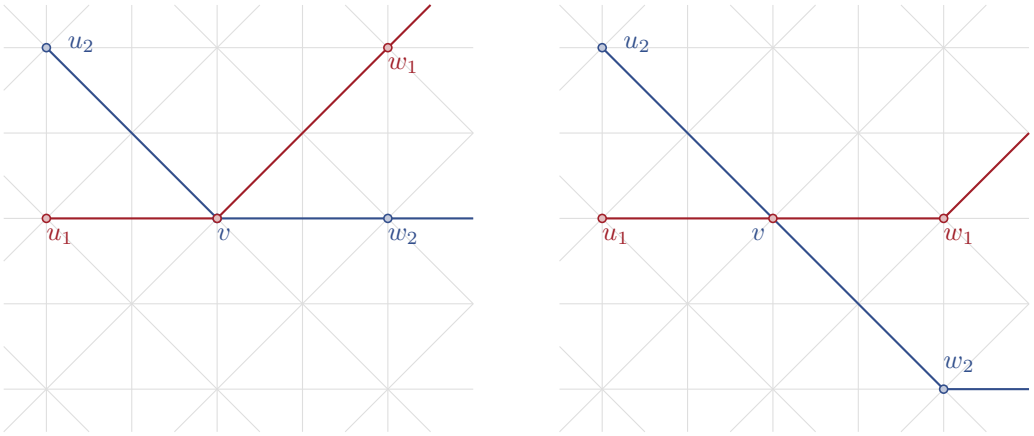
Relative positions The soft clauses for relative positions get weight f_2:

$$\neg\xi(u,v)^i \qquad \forall(u,v) \in E,\ \forall i : 1 \leq i \leq 4 \tag{27.1}$$

Compactness The soft clauses for compactness are added with weight f_3:

$$\neg\lambda(u,v)^i \qquad \forall(u,v) \in E,\ \forall i : L_{min}(u,v) < i \leq L_{min}(u,v) + 4 \text{ if } deg(u) > 1 \wedge deg(v) > 1 \tag{28.1}$$

For better efficiency, we omit these constraints for edges to end stations to replace them with fixed lengths (the minimal length of the edge) as explained beforehand in Subsection 3.4.3.



(a) Line straightness in stations with $deg \leq 2$ (b) Line straightness in stations with $deg > 2$

Figure 3.5: Line straightness in comparison. It can make a difference to prefer line straightness in stations with different numbers of edges. One can compare the two drawings in (a) and (b) that match in edge length and number of bends, but differ in the stations the bends are placed on. The preferred kind of placing these bends is a matter of opinion and can even depend on the input map. Note that the two drawings also match in relative positions if we assume that the edges (v, w_1) and (v, w_2) both have the original sector 0.

CHAPTER 4

SMT model

The satisfiability problem over boolean formulas in all its variations as described at the beginning of Chapter 3 can be generalized to check satisfiability of any type of mathematical formulas. With satisfiability modulo theories (SMT) [BSST21] mathematical expressions using various data types as integers or real numbers can be interpreted within any background theory. This theory defines how different predicates or functions are interpreted when checking satisfiability of an input formula. An example for such a theory is integer arithmetic, that defines usual mathematical expressions as given below.

$0, 1, 2, \ldots$	constants
$+$	Addition
$-$	Subtraction
$=$	Equality
$<$	Linear ordering
max	Maximum
min	Minimum
$\ldots$	$\ldots$

Table 4.1: Integer arithmetic as theory.

An extension to real number arithmetics is easily possible. Number arithmetics extended with boolean arithmetics, regular expressions and functional arrays are the standard theories used for SMT, depending on the data types used. Note that integer variables (or other numeric variables) can be unbounded in SMT. The same holds for other data types as arrays or strings.

There exist dedicated SMT solvers, which are able to read the standard format SMT-LIB [BFT16]. Expressions in this format are also called s-expressions and use a Lisp-like [McC78] format to represent literals or constraints as formatted string.

The s-expression format can be used to write conjunctions and disjunctions as known from SAT. A conjunction $a \wedge b$ and a disjunction $a \vee b$ are written in s-expression as (29.1) and (29.1). Note that a and b can be literals as in SAT, or any other s-expression that has a boolean result, like conjunctions or (in)equalities.

```
(and |a| |b|)    (29.1)
(or |a| |b|)     (29.2)
```

In correct s-expression syntax, for (in)equalities like $a = b$ or $a < b$ a binary relation as in (30.1) or (30.2) would be used. Also for negations $\neg\alpha$ of an s-expression α a unary relation as in (30.3) would be used. The same holds for additions as can be seen in (30.4) for $a + 1$.

```
(= |a| |b|)      (30.1)
(< |a| |b|)      (30.2)
(not |α|)        (30.3)
(add |a| 1)      (30.4)
```

We will write negations, (in)equalities and additions as $\neg a$, $a = b$, $a < b$ and $a + 1$ as a shorthand to provide better readability. One should keep in mind that these expressions and any of their combinations can be written as proper s-expression

4.1 Variable encoding

A benefit of using an SMT solver in comparison to SAT solvers is that we can use other data types than only boolean variables. For schematic metro maps we can use integer values everywhere we used unary encoding with SAT. It is also possible to form (in)equalities with these variables and use them as boolean statements. If an (in)equality can be fulfilled, then the boolean statement is interpreted as *true*, if it is violated, then it is interpreted as *false*. These boolean statements can be used as literals in clauses, but clauses are called constraints in SMT. In fact a constraint in SMT does not have to be a disjunction of boolean statements, it can have nested conjunctions as well.

To encode an integer variable in SMT, we only have to declare it as variable of data type integer. For an integer variable a this would look like (31.1) in s-expression format.

```
(declare-fun |a| () Int)    (31.1)
```

In fact a variable in SMT is declared as function without parameters.

4.2 Coordinates

The coordinate variables for the L^∞-metric are declared as integer variables as well as the direction values as shown in (32.1) to (32.8).

$$\texttt{(declare-fun } |x(v)| \texttt{ () Int)} \qquad \forall v \in V \qquad (32.1)$$
$$\texttt{(declare-fun } |y(v)| \texttt{ () Int)} \qquad \forall v \in V \qquad (32.2)$$
$$\texttt{(declare-fun } |z_1(v)| \texttt{ () Int)} \qquad \forall v \in V \qquad (32.3)$$
$$\texttt{(declare-fun } |z_2(v)| \texttt{ () Int)} \qquad \forall v \in V \qquad (32.4)$$
$$\texttt{(declare-fun } |z_3(v)| \texttt{ () Int)} \qquad \forall v \in V \qquad (32.5)$$
$$\texttt{(declare-fun } |z_4(v)| \texttt{ () Int)} \qquad \forall v \in V \qquad (32.6)$$
$$\texttt{(declare-fun } |dir(u,v)| \texttt{ () Int)} \qquad \forall (u,v) \in E \qquad (32.7)$$
$$\texttt{(declare-fun } |dir(v,u)| \texttt{ () Int)} \qquad \forall (u,v) \in E \qquad (32.8)$$

Constraints (33.1) to (33.10) model all (existing) upper and lower bounds for each of the previously defined variables. Note that it is not necessary to find upper bounds for the coordinates of each station in SMT.

$$\texttt{(assert } |x(v) \geq 0|\texttt{)} \qquad \forall v \in V \qquad (33.1)$$
$$\texttt{(assert } |y(v) \geq 0|\texttt{)} \qquad \forall v \in V \qquad (33.2)$$
$$\texttt{(assert } |z_1(v) \geq 0|\texttt{)} \qquad \forall v \in V \qquad (33.3)$$
$$\texttt{(assert } |z_2(v) \geq 0|\texttt{)} \qquad \forall v \in V \qquad (33.4)$$
$$\texttt{(assert } |z_3(v) \geq 0|\texttt{)} \qquad \forall v \in V \qquad (33.5)$$
$$\texttt{(assert } |z_4(v) \geq 0|\texttt{)} \qquad \forall v \in V \qquad (33.6)$$
$$\texttt{(assert } |dir(u,v) \geq \min\{S(u,v)\}|\texttt{)} \qquad \forall (u,v) \in E \qquad (33.7)$$
$$\texttt{(assert } |dir(u,v) \leq \max\{S(u,v)\}|\texttt{)} \qquad \forall (u,v) \in E \qquad (33.8)$$
$$\texttt{(assert } |dir(v,u) \geq \min\{S(u,v)\}|\texttt{)} \qquad \forall (u,v) \in E \qquad (33.9)$$
$$\texttt{(assert } |dir(v,u) \leq \max\{S(u,v)\}|\texttt{)} \qquad \forall (u,v) \in E \qquad (33.10)$$

We use a tighter bound for the direction values than $0 \leq dir(u,v) < 8$ because per definition of $S(u,v)$ we know that $0 \leq i < 8$ holds for all $i \in S(u,v)$ and if $j \notin S(u,v)$ then $dir(u,v) = j$ can not hold. Tighter bounds reduce the number of possibilities the SMT solver has to check, which can lead to better efficiency.

The constraints in (34.1) to (34.4) model the modified L^∞-metric as defined in Section 3.2.

$$\texttt{(assert } |z_0(v) = 2x(v)|\texttt{)} \qquad (34.1)$$
$$\texttt{(assert } |z_1(v) = x(v) + y(v)|\texttt{)} \qquad (34.2)$$
$$\texttt{(assert } |z_2(v) = 2y(v)|\texttt{)} \qquad (34.3)$$
$$\texttt{(assert } |z_3(v) = y(v) - x(v)|\texttt{)} \qquad (34.4)$$

4.3 Hard constraints

Using integer variables with (in)equalities as boolean statements, we can use most of the hard constraints defined in (11.1)-(23.4) as they are, because they can be defined in SMT like this. In this section we give all used hard constraints in their used version for SMT and state the differences to SAT.

4.3.1 Edge directions and minimum length

We introduce the same set of boolean values $\alpha_i(u,v)$ for each edge $(u,v) \in E$ and each $i \in S(u,v)$.

$$\texttt{(declare-fun |}\alpha_i(u,v)\texttt{| () Bool)} \qquad \forall (u,v) \in E, \forall i \in S(u,v)$$

The clauses (35.1) to (35.2) are the same as with SAT (but written in s-expression syntax). In (36.1) to (36.5) the translation of (in)equalities into unary encoding is not necessary.

$\forall (u,v) \in E$:

$$\texttt{(assert (or |}\alpha_{i_1}(u,v)\texttt{| } \cdots \texttt{))} \qquad \text{with } i_{1\ldots} \in S(u,v) \tag{35.1}$$

$$\texttt{(assert (or |}\neg\alpha_i(u,v)\texttt{| |}\neg\alpha_j(u,v)\texttt{|))} \qquad \forall i,j : i < j \in S(u,v) \tag{35.2}$$

$\forall (u,v) \in E, \forall i \in S(u,v)$:

$$\texttt{(assert (or |}\neg\alpha_i(u,v)\texttt{| |}dir(u,v) = i\texttt{|))} \tag{36.1}$$

$$\texttt{(assert (or |}\neg\alpha_i(u,v)\texttt{| |}dir(v,u) = (i+4) \bmod 8\texttt{|))} \tag{36.2}$$

$$\texttt{(assert (or |}\neg\alpha_i(u,v)\texttt{| |}z_{i^o}(u) = z_{i^o}(v)\texttt{|))} \tag{36.3}$$

$$\texttt{(assert (or |}\neg\alpha_i(u,v)\texttt{| |}z_i(u) + 2L_{min}(u,v) \leq z_i(v)\texttt{|))} \qquad \text{if } i < 4 \tag{36.4}$$

$$\texttt{(assert (or |}\neg\alpha_i(u,v)\texttt{| |}z_{i-4}(v) + 2L_{min}(u,v) \leq z_{i-4}(u)\texttt{|))} \qquad \text{if } i \geq 4 \tag{36.5}$$

Note that $i^o = (i+2) \mod 4$.

We omit the correct s-expression syntax for (in)equalities, negation and addition here as stated in Section 4.1 and we will do the same in the following sections.

4.3.2 Combinatorial embedding / Circular vertex orders

We introduce the set of boolean variables $(\beta_1(v), \ldots, \beta_{deg(v)}(v))$ containing one variable for each neighbour of v for each vertex $v \in V$ with more than two neighbours.

$$\texttt{(declare-fun |}\beta_i(v)\texttt{| () Bool)} \qquad \forall v \in V : deg(v) > 2, \forall 1 \leq i \leq deg(v)$$

Recall that $(u_1, \ldots, u_{deg(v)})$ are the circular ordered neighbors of vertex v.

$\forall v \in V : deg(v) > 2 :$

$$\texttt{(assert (or } |\beta_1(v)| \cdots |\beta_{deg(v)}(v)| \texttt{))} \tag{37.1}$$

$$\texttt{(assert (or } |\neg\beta_i(v)| \; |\neg\beta_j(v)| \texttt{))} \qquad \forall i,j : 1 \leq i < j \leq deg(v) \tag{37.2}$$

$$\texttt{(assert (or } |\beta_i(v)| \; |dir(v,u_i) + 1 \leq dir(v,u_{i+1})| \texttt{))} \qquad \forall i : 1 \leq i < deg(v) \tag{37.3}$$

$$\texttt{(assert (or } |\beta_i(v)| \; |dir(v,u_i) + 1 \leq dir(v,u_1)| \texttt{))} \qquad i = deg(v) \tag{37.4}$$

$\forall v \in V : deg(v) = 2 :$

$$\texttt{(assert } |dir(v,u_1) \neq dir(v,u_2)| \texttt{)} \tag{38.1}$$

Again (37.1) to (37.2) are the same constraints as used with SAT in s-expression syntax. Constraints (37.3) to (38.1) with inequalities do not need the transformation into unary encoding.

4.3.3 Planarity / Edge spacing

We define the set of edge pairs E' as stated in Subsection 3.3.3. For each pair of edges $(e, e') \in E'$ we introduce the set of boolean variables $(\gamma_0(e,e'), \ldots, \gamma_7(e,e'))$ to indicate the direction in which the two edges e and e' are separated.

$$\texttt{(declare-fun } |\gamma_i(e,e')| \texttt{ () Bool)} \qquad \forall (e,e') \in E', \forall i : 0 \leq i < 8$$

The constraints (39.1) to ensure at least one variable from each set is *true* are the same as used in SAT (again in s-expression syntax).

$$\texttt{(assert (or } |\gamma_0(e,e')| \cdots |\gamma_7(e,e')| \texttt{))} \qquad \forall (e,e') \in E' \tag{39.1}$$

As before the transformation with unary encoding is not needed for the inequalities in (40.1) to (41.4).

$\forall (e = (u,v), e' = (u',v')) \in E', \forall i \in \mathbb{N} : 0 \leq i < 4 :$

$$\texttt{(assert (or } |\neg\gamma_i(e,e')| \; |z_i(u) + d_{min} \leq z_i(u')| \texttt{))} \tag{40.1}$$

$$\texttt{(assert (or } |\neg\gamma_i(e,e')| \; |z_i(v) + d_{min} \leq z_i(u')| \texttt{))} \tag{40.2}$$

$$\texttt{(assert (or } |\neg\gamma_i(e,e')| \; |z_i(u) + d_{min} \leq z_i(v')| \texttt{))} \tag{40.3}$$

$$\texttt{(assert (or } |\neg\gamma_i(e,e')| \; |z_i(v) + d_{min} \leq z_i(v')| \texttt{))} \tag{40.4}$$

$\forall (e = (u,v), e' = (u',v')) \in E', \forall i \in \mathbb{N} : 0 \leq i < 4 :$

$$\texttt{(assert (or } |\neg\gamma_{i+4}(e,e')| \; |z_i(u') + d_{min} \leq z_i(u)| \texttt{))} \tag{41.1}$$

$$\texttt{(assert (or } |\neg\gamma_{i+4}(e,e')| \; |z_i(u') + d_{min} \leq z_i(v)| \texttt{))} \tag{41.2}$$

$$\texttt{(assert (or } |\neg\gamma_{i+4}(e,e')| \; |z_i(v') + d_{min} \leq z_i(u)| \texttt{))} \tag{41.3}$$

$$\texttt{(assert (or } |\neg\gamma_{i+4}(e,e')| \; |z_i(v') + d_{min} \leq z_i(v)| \texttt{))} \tag{41.4}$$

The number of constraints used for planarity in SMT is still the highest with respect to big O notation. The number is quadratic in the number of vertices and dominates the number of all other constraints which are linearly many. Hence we also use these constraints only on demand. This means we calculate a solution without them and check for planarity. If planarity is violated, we add the corresponding constraints for the violating edges and recalculate a new solution exactly as we do with SAT.

4.4 Optimization

There is no difference between SAT and SMT in the pattern of constraint generation regarding optimization and the differentiation of hard and soft constraints. Hence we add the equivalent hard constraints we added in SAT to measure the optimized values as well as the corresponding soft constraints.

4.4.1 Line bends

We declare the integer variable $\theta(u, v, w)$ for each pair of consecutive edges (u, v) and (v, w) on some path $L \in \mathcal{L}$, to store the gap between the directions $dir(u, v)$ and $dir(v, w)$.

$$\texttt{(declare-fun } |\theta(u,v,w)| \texttt{ () Int)} \qquad \forall L \in \mathcal{L},\ \forall (u,v),(v,w) \in L$$

The bounds $0 \leq \theta(u, v, w) < 4$ are set as well.

$$\texttt{(assert } |\theta(u,v,w) \leq 3|\texttt{)} \qquad \forall L \in \mathcal{L},\ \forall (u,v),(v,w) \in L \tag{42.1}$$
$$\texttt{(assert } |\theta(u,v,w) \geq 0|\texttt{)} \qquad \forall L \in \mathcal{L},\ \forall (u,v),(v,w) \in L \tag{42.2}$$

We also introduce the two binary correction variables $\delta_1(u, v, w)$ and $\delta_2(u, v, w)$ for each of these edge pairs.

$$\texttt{(declare-fun } |\delta_1(u,v,w)| \texttt{ () Bool)} \qquad \forall L \in \mathcal{L},\ \forall (u,v),(v,w) \in L$$
$$\texttt{(declare-fun } |\delta_2(u,v,w)| \texttt{ () Bool)} \qquad \forall L \in \mathcal{L},\ \forall (u,v),(v,w) \in L$$

The constraints in (43.1) to (43.4) are the SMT version of the constraints given in (18.1) to (18.4) without the translation step into (19.1) to (19.4), because with SMT it is more complex to split the inequalities into bitwise constraints, than using inequalities with more than two variables.

$\forall L \in \mathcal{L},\ \forall (u, v), (v, w) \in L$:

$$\texttt{(assert (or } |\delta_2(u,v,w)| \ |\theta(u,v,w) \geq dir(v,w) - dir(u,v)|\texttt{))} \tag{43.1}$$
$$\texttt{(assert (or } |\delta_1(u,v,w)| \ |\theta(u,v,w) \geq dir(u,v) - dir(v,w)|\texttt{))} \tag{43.2}$$
$$\texttt{(assert (or } |\neg\delta_1(u,v,w)| \ |\theta(u,v,w) \geq dir(v,w) - dir(u,v) + 8|\texttt{))} \tag{43.3}$$
$$\texttt{(assert (or } |\neg\delta_2(u,v,w)| \ |\theta(u,v,w) \geq dir(u,v) - dir(v,w) + 8|\texttt{))} \tag{43.4}$$

Because of the transformation in SAT using unary encoding, a lot more clauses were necessary than we have constraints in SMT. The difference is up to a factor of 8, because that's the range of the direction variables, multiplied by the factor of 4, because that is the range of the θ variables. This is also the case with all other constraints that need unary encoding in SAT.

4.4.2 Relative positions

We declare the integer variable $\xi(u, v)$ for each edge (u, v) to measure the gap between the direction $dir(u, v)$ in the drawing and the original section of the edge $sec_u(v)$.

$$\texttt{(declare-fun |}\xi(u,v)\texttt{| () Int)} \qquad \forall (u,v) \in E$$

Again we add the bounds $0 \leq \xi(u, v) \leq dev$, because dev is the maximal deviation of sections from the original input that we allow.

$$\texttt{(assert |}\xi(u,v) \leq dev\texttt{|)} \qquad \forall (u,v) \in E \tag{44.1}$$
$$\texttt{(assert |}\xi(u,v) \geq 0\texttt{|)} \qquad \forall (u,v) \in E \tag{44.2}$$

We also introduce the two binary correction variables $\eta_1(u, v)$ and $\eta_2(u, v)$ for each edge.

$$\texttt{(declare-fun |}\eta_1(u,v)\texttt{| () Bool)} \qquad \forall (u,v) \in E$$
$$\texttt{(declare-fun |}\eta_2(u,v)\texttt{| () Bool)} \qquad \forall (u,v) \in E$$

Again as with line bends we can work with the first version of the constraints generated in the SAT section (20.1)-(20.4), hence we end up with the constraints in (45.1)-(45.4). That is still because we do not gain complexity with more summands per inequality in SMT.

$\forall (u, v) \in E$:

$$\texttt{(assert (or |}\eta_2(u,v)\texttt{| |}sec_u(v) - dir(u,v) \leq \xi(u,v)\texttt{|))} \tag{45.1}$$
$$\texttt{(assert (or |}\eta_1(u,v)\texttt{| |}dir(u,v) - sec_u(v) \leq \xi(u,v)\texttt{|))} \tag{45.2}$$
$$\texttt{(assert (or |}\neg\eta_1(u,v)\texttt{| |}sec_u(v) - dir(u,v) + 8 \leq \xi(u,v)\texttt{|))} \tag{45.3}$$
$$\texttt{(assert (or |}\neg\eta_2(u,v)\texttt{| |}dir(u,v) - sec_u(v) + 8 \leq \xi(u,v)\texttt{|))} \tag{45.4}$$

Note that the number of constraints in comparison to the SAT implementation only differs by a factor of dev, that's the range of the ξ variables, because unary encoding was not necessary with SAT here. This means in SAT we do have significantly less clauses for the relative positions per $\xi(u, v)$ variable than we had for line bends per $\theta(u, v, w)$ variable. In SMT we have the same number of clauses per variable for both measurements.

4.4.3 Compactness

Using the L^∞-metric, the length of an edge (u, v) is the maximum distance of u and v on the x-axis and the y-axis. We introduce the integer variable $\lambda(u, v)$ for each edge (u, v) to measure the distance between u and v.

$$\texttt{(declare-fun |}\lambda(u,v)\texttt{| () Int)} \qquad \forall(u,v) \in E$$

The edge length is only lower bounded by the minimal length $L_{min}(u, v)$ of the edge (u, v). We do not use an upper bound.

$$\texttt{(assert |}\lambda(u,v) \geq L_{min}(u,v)\texttt{|)} \qquad \forall(u,v) \in E \tag{46.1}$$

Again we can utilize the properties of the L^∞-metric to generate the constraints (47.1) to (48.1) in the same way as in SAT without the necessity of unary encoding.

$\forall(u, v) \in E$:

$$\texttt{(assert |}x(u) - x(v) \leq \lambda(u,v)\texttt{|)} \tag{47.1}$$
$$\texttt{(assert |}x(v) - x(u) \leq \lambda(u,v)\texttt{|)} \tag{47.2}$$
$$\texttt{(assert |}y(u) - y(v) \leq \lambda(u,v)\texttt{|)} \tag{47.3}$$
$$\texttt{(assert |}y(v) - y(u) \leq \lambda(u,v)\texttt{|)} \tag{47.4}$$

For the optimization gain as described in the SAT section with Figure 3.4 we fix the edge length of edges to terminal stations to the minimal length of the edge $L_{min}(u, v)$.

$\forall(u, v) \in E :$ if $deg(u) = 1 \vee deg(v) = 1$:

$$\texttt{(assert |}\lambda(u,v) = L_{min}(u,v)\texttt{|)} \tag{48.1}$$

4.4.4 Soft constraints

The soft constraints are also handled similar to SAT, but we use the inequalities that are encoded by the boolean variables directly. To minimize a value x we add the constraints that x has to be smaller than i for each $l_x < i \leq u_x$ with l_x being the lower bound and u_x the upper bound of the variable.

Line bends $\forall L \in \mathcal{L},\ \forall(u, v), (v, w) \in L,\ \forall i \in \mathbb{N} : 1 \leq i < 4 :$

$$\texttt{(assert-soft |}\theta(u,v,w) < i\texttt{| :weight } f_1\texttt{)} \tag{49.1}$$

Relative positions $\forall(u, v) \in E,\ \forall i \in \mathbb{N} : 1 \leq i \leq 4 :$

$$\texttt{(assert-soft |}\xi(u,v) < i\texttt{| :weight } f_2\texttt{)} \tag{50.1}$$

Compactness $\forall (u,v) \in E,\ \forall i \in \mathbb{N} : L_{min}(u,v) < i \leq L_{min}(u,v) + 4 :$

$$\texttt{(assert-soft } |\lambda(u,v) < i| \texttt{ :weight } f_3\texttt{)} \tag{51.1}$$

Note that the edge length is not bounded, but we consider the edges with length greater than the minimum length plus four as long edges all incurring the same cost. This removes the ability to distinguish between edges which exceed length four, however such edges do only appear rarely in practical examples. The bound of length four was determined experimentally.

CHAPTER 5

Experiments

In this section we evaluate our models by running experiments on standard benchmark instances taken from the literature. This is done in order to evaluate their performance against already existing models.

5.1 Input instances

We evaluated our models on different real world instances of metro networks. For that we used the geographical metro networks of different cities as input instances. We used standard benchmarks that are real world metro networks of different cities. These cities are Vienna, Karlsruhe, Lisbon, Montreal and Washington. The input instances are considered to be graphs that are given as `.graphml` files with specified x and y coordinates corresponding to the actual geographic location of each station. The edges between stations are straight line segments.

	Vienna		Karlsruhe		Lisbon		Montreal		Washington		
	a/b	c	a/b	c	a/b	c	a/b	c	a	b	c
vertices	90	44	126	70	60	49	65	26	97	98	47
edges	96	50	132	76	72	61	66	27	99	101	50
lines	5		10		9		4		6		
max lines/edge	1		8		2		1		3		
max deg	5		5		5		5		5		

Table 5.1: Characteristics of the used input maps. a: Original input map. b: Maps after planarization. This is the same as the original input for input maps with a planar embedding. c: Maps after the two-degree heuristic (with the planarization beforehand).

In Table 5.1 we give a small analysis of all used input instances. It contains the number of vertices in the input graph as well as the number of edges. Beside that we count the

number of metro lines in the whole metro network. The maximal number of metro lines per edge gives some sense of the complexity of the network just like the maximal degree over all vertices. Another indicator of map complexity would be the difference between the number of vertices and the number of edges, this value can be easily calculated from the table as well.

Beside these analytics on the original graph, we give these values also for the graph resulting from the planarization step as well as for the graph resulting from the two degree heuristic (both as defined in Section 2.3) afterwards. The last version of the instance, after the planarization and the two degree heuristic are applied, is the version of the instance that is used as input for our model calculations.

In terms of vertices, the smallest input map is the metro map of Montreal. The biggest one is the metro map of Karlsruhe, in terms of vertices, but also as for the number of metro lines in the whole network. The map of Lisbon has only one metro line less than Karlsruhe, but its metro line connections are less complicated because Karlsruhe also has the most metro lines per single edge. Lisbon is still a complicated map, because it has the most edges relatively to the number of vertices. In Section 5.4 we will have a look at the correlation of these properties with the solving times of our models.

	weights			$L_{min}(u,v)$	d_{min}
	f_1	f_2	f_3		
Instance 1	3	2	1	1	1
Instance 2	5	2	1		
Instance 3	6	4	2	or modified by deg-2	

Table 5.2: Input parameters.

As seen in Section 2.2 there are additional input parameters for the schematic metro map problem beside the metro networks itself. For each city network that we use as input, we generate 3 independent instances. Their values in our experiments are given in Table 5.2. Each instance has a different vector as input weights. These weights are $(3, 2, 1)$, $(5, 2, 1)$ and $(6, 4, 2)$. Note that instances with weights $(3, 2, 1)$ and $(6, 4, 2)$ represent the same importance of soft constraints, because they are relatively the same. This means the results should be the same, or have at least the same valence with respect to the cost function. The parameter d_{min} is the same for all instances and has the default value $d_{min} = 1$.

As we have metro maps of five different cities and three different weight vectors for each of them, we used 15 unique instances.

5.2 Software implementation

Our model is implemented using the programming language `python` with version 3.6.9. For the implementation we need several `python` packages as interface to the used

solvers. The first one used is the PySAT package `python-sat` [IMM18]. It is a toolkit to use different SAT solvers in python. The SAT solver we use is the G3 sovler. To achieve the MaxSAT functionality we used the Relaxable Cardinality Constraints (RC2) [MDM14, MIM14] implementation of the PySAT example package and extended it to support timeouts.

The second relevant python package is the Z3Py package `z3-solver` [dMBWN]. It is a wrapper for the SMT solver $Z3$ implemented by Microsoft Research. For the MaxSMT problem the Z3Py package provides the νZ module that is an extension to the $Z3$ solver that allows optimization with SMT.

An alternative for MaxSAT would have been the Linear (search) SAT-UNSMT (LSU) [MHL+13] implementation from the PySAT example package. We did not use that, because it does not have the possibility to add constraints on demand. This is a feature that is necessary for our implementation to add the planarity constraints as described in the previous chapters just in case they are needed. If a solver does not support this feature, then it has to be restarted instead. This would wipe all knowledge gained from other constraints until this point, such that the same knowledge has to be recalculated again after adding a new constraint. A corresponding feature for solvers is the extraction of partial solutions. This means, that it is possible to handle none-optimal solutions found by the solver during the calculation process at runtime via some callback function. This is a feature that is commonly supported by MIP solvers like CPLEX and Gurobi. In extension to this they do not only give the opportunity to handle non-optimal solutions, but also even infeasible solutions in terms of solutions using real numbers for variables that should be integers. This can be very helpful when adding constraints on demand, because we do not have to wait until the solver optimizes the solution, when we already know that the solution is likely to be infeasible and instead adding the additional constraints at runtime via the callback function. The $Z3$ solver implemented this feature with non-optimal solutions for the νZ module in version 4.9.0. The MaxSAT solvers that we had available do not provide this feature. This has to be taken into account when comparing runtime results. Another benefit of extraction of partial solutions is to keep track of the best solution so far. This is helpful in combination with timeouts, because we can extract at least some solution if the instance takes too long to solve optimally.

Another helpful python package is the `pickle` package [VR20] to export resulting maps. We use this export to save the data of maps with coordinates for each station that can be analyzed again later on. The second exporter used is `ipey` [Wal], an interface to generate and modify Ipe files. Ipe [Che22] is an extensible drawing editor with LaTeX integration and gives the possibility to generate figures and presentations in PDF format. We use the Ipe file type to render the resulting schematic metro maps. [1]

[1] In fact the Ipe extensible drawing editor [Che22] is used to create nearly all figures in this thesis.

5.3 Hardware specification

The computation cluster used for our experiments consists of 16 nodes equipped with dual Xeon E5-2640 v4 (10-core 2,40 GHz) CPUs and 160 GB of DDR4 Memory. Considering the 10 cores per CPU each node has 20 physical cores which leads to a theoretical limit of 20 concurrent jobs being executed per node. As we use 32GB of RAM per job we only can schedule 5 jobs per cluster node without running into memory bottlenecks which further would decrease the performance because of swapping. In conclusion this leads to 80 jobs running for our studies parallel at max in the whole cluster in theory. Practically there is memory overhead on the nodes that allows us to schedule at most 4 jobs that can be run in parallel per cluster node. Hence there can be only 64 jobs running per cluster node without memory bottlenecks.

Even if multithreading would be possible on the computation cluster, we deactivated it for our experiments and only used one thread for each instance. We did this to make results more comparable, because runtimes can depend on the multithreading abilities of the different solvers and this could distort runtime results.

5.4 Runtimes

To compare the efficiency of our encoding with previous results of the MIP encoding of Nöllenburg and Wolff [NW10] we ran experiments on the computation cluster. We started 20 separate runs for each instance described in Section 5.1 on the MIP solver as well as on the SAT and SMT solvers. When not stated otherwise while mentioning runtimes we talk about average runtimes over all these 20 runs. With 15 unique instances on three different solvers with 20 runs each, we had to run 900 jobs on the computation cluster. Since we know that the problem we try to solve is NP-hard, the solution times could possibly be very excessive for each instance. We set a runtime limit of 36000 seconds (10 hours), what lead to a cumulative runtime of up to 9000 hours, which corresponds to 375 full days. Taking into account the 64 parallel running jobs we could have ended up with nearly 6 days computation time in a worst case scenario where no instance can be solved within our set timeout. Luckily solutions for the small metro maps like Montreal or Vienna could be computed in some minutes or at most around half a dozen hours. This decreased our real computation time enormously. In fact we had an overall runtime of 12577680 seconds, hence with parallel jobs taken into account it took the cluster around 2.3 days to finish all tasks.

The results for all 20 runs on each instance are given in Table 5.3. The first five columns give the specifications of the different instances. The column `runtimes` gives the average runtime over all 20 instances on the same input instance. Via the column `solutions` indicates whether any solutions were found during the 20 different runs for each instance. The SAT instances for Karlsruhe and Lisbon did not find solutions at all before running into the configured timeout of 10 hours. All other instances produced at least some solutions within the runtime limit. Whether the solution found was optimal or not is

city	f1	f2	f3	algo	runtime	solutions	opt. sol.
Karlsruhe	3	2	1	mip	36000	✓	?
				sat	36000	×	-
				smt	36000	✓	×
	5	2	1	mip	36000	✓	?
				sat	36000	×	-
				smt	36000	✓	×
	6	4	2	mip	36000	✓	?
				sat	36000	×	-
				smt	36000	✓	×
Lissabon	3	2	1	mip	6866	✓	✓
				sat	36000	×	-
				smt	7983	✓	✓
	5	2	1	mip	7635	✓	✓
				sat	36000	×	-
				smt	6955	✓	✓
	6	4	2	mip	3328	✓	✓
				sat	36000	×	-
				smt	11358	✓	✓
Montreal	3	2	1	mip	11	✓	✓
				sat	24	✓	✓
				smt	200	✓	✓
	5	2	1	mip	9	✓	✓
				sat	12	✓	✓
				smt	72	✓	✓
	6	4	2	mip	10	✓	✓
				sat	23	✓	✓
				smt	75	✓	✓
Washington	3	2	1	mip	778	✓	✓
				sat	3147	✓	✓
				smt	36000	✓	✓
	5	2	1	mip	266	✓	✓
				sat	2818	✓	✓
				smt	4787	✓	✓
	6	4	2	mip	202	✓	✓
				sat	3055	✓	✓
				smt	*33023	✓	*✓
Vienna	3	2	1	mip	465	✓	✓
				sat	13456	✓	✓
				smt	5538	✓	✓
	5	2	1	mip	177	✓	✓
				sat	8738	✓	✓
				smt	9742	✓	✓
	6	4	2	mip	1130	✓	✓
				sat	14107	✓	✓
				smt	14894	✓	✓

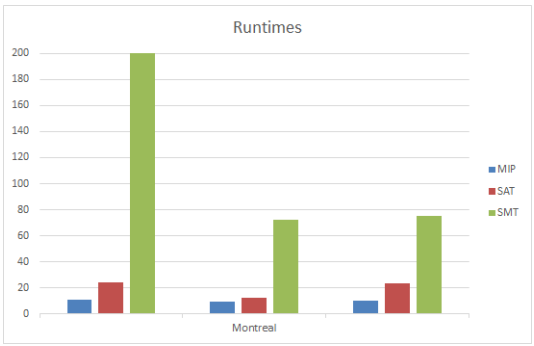
Table 5.3: Result overview for 20 runs each. Runtime in seconds: average over all 20 runs.
Solutions: ✓ solution found in every run, × no solution found at all.
Opt. sol.: ✓ optimal solution found, ? optimality of the solution was not proven and is unknown, × optimal solution was not found.

given in column `opt. sol.`. A checkmark indicates that the solutions were proven optimal. A cross indicates the solution was definitely not optimal. This is only the case for the instances of Karlsruhe and we know the fact that the solution is not optimal from comparison with solutions of the MIP solver. The last possible indicator in this column is a question mark that points out that we do not know if the solution is optimal or not, because there is no other solver that found a better solution and optimality could not be proven within the runtime limit. The solutions found in different runs on the same instances were alwas the same for all 20 runs with one exception. On the Washington instance with weights $(6, 4, 2)$ the SMT solver found and proved the optimal solution three times, but had a timeout without finding the optimal solution the other 17 times. This instance is marked with an asterisk in the table. Additionally the runtimes are represented in Figure 5.1.

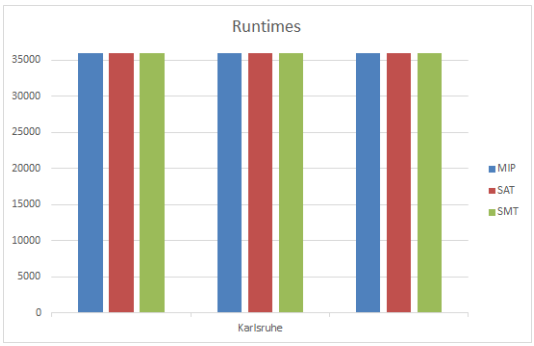
One can observe that for the biggest map Karlsruhe no solver could find and prove the optimal solution within the given runtime limit. Note that it is still possible that the optimal solution was found by some solver but the solver could not prove if it is optimal or not yet. The SAT solver did not yield any solution. Remember that it is technically not possible to extract an intermediate solution from the used solver with the `pysat` library. This means maybe the SAT solver found the optimal solution or at least a quite good one, but we can not tell if this is the case or not. The SMT and MIP solvers generated intermediate solutions. As shown in Table 5.4 the MIP solver found better solutions as the SMT solver for each weight vector This means the SMT solution is definitely not optimal, but we can not tell anything about the MIP solution.

For the metro map of Lisbon the SAT solver did also not find the optimal solution within the given runtime limit. Again we did not get a non-optimal solution as well, because of the lack of support for intermediate solutions. the MIP and SMT solvers both found the optimal solution. We could not observe that one of them was strictly faster than the other one, because on the instance with weight vector $(5, 2, 1)$ SMT was faster but for the other two instances MIP was faster on average. For the $(3, 2, 1)$ and $(6, 4, 2)$ instances here we observed interesting runtime discrepancies. In theory these two instances are equivalent as they should produce the same output w.r.t the objective function because the relative differences between the weights are the same, with the only difference that the second instance should have doubled cost of the first one. Even if the solutions generated were the same and MIP was faster than SMT for both, the runtimes differed significantly. MIP was faster on the instance with weights $(6, 4, 2)$ than with $(3, 2, 1)$ while SMT was significantly faster with weights $(3, 2, 1)$ than with $(6, 4, 2)$. This is an effect, which we can not explain.

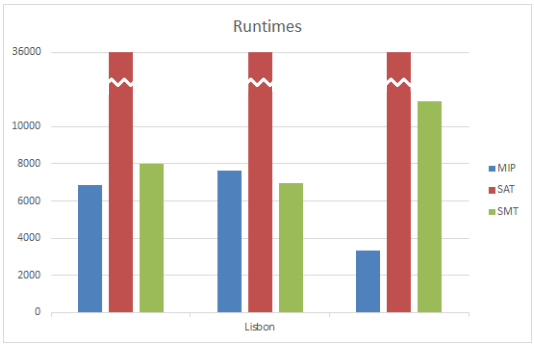
On the Montreal metro map, which was the smallest input instance, MIP had nearly no deviation between $(3, 2, 1)$ and $(6, 4, 2)$ while the same holds for SAT. SMT in comparison had quite longer runtimes for the $(3, 2, 1)$ instance. That is interesting, because we already saw discrepancies like that for Lisbon, but there it was the other way around for SMT. All solvers had slightly faster results for the instance with weights $(5, 2, 1)$. Note that we are talking about deviations within at most 200 seconds for Montreal, hence the



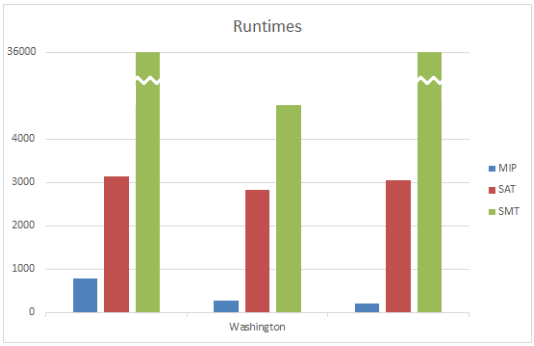
(a) Runtimes for Montreal instances.



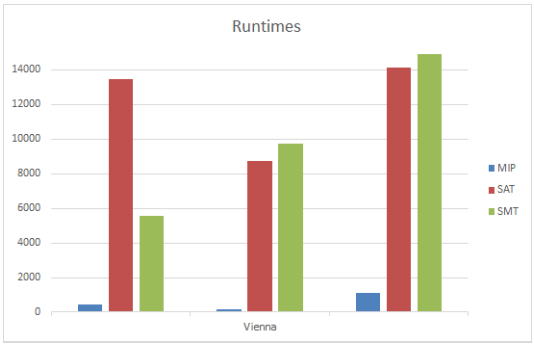
(b) Runtimes for Karlsruhe instances.



(c) Runtimes for Lisbon instances.



(d) Runtimes for Washington instances.



(e) Runtimes for Vienna instances.

Figure 5.1: Runtimes in seconds for different input instances. For each country the instances from left to right have weight vectors $(3, 2, 1)$, $(5, 2, 1)$ and $(6, 4, 2)$.

runtime differences can easily be caused by some initialization overhead.

The metro map of Washington and it's results are worth some more discussion. Concerning runtimes we could observe huge discrepancies between instances with different weights for SMT. For $(5, 2, 1)$ the solver found the optimal solution within 47 minutes, but for the other two instances the process run into timeouts. In more detail for $(3, 2, 1)$ all 20 runs had timeouts after the limit of 10 hours, while for $(6, 4, 2)$ the solver found the optimal solution within 4.5 hours in 3 runs, but had timeouts for all other runs. In comparison to that the MIP and SAT solvers did not show discrepancies like that. Both found optimal solutions, MIP within around 200 to 800 seconds and SAT within around 3000 seconds. The only discrepancies we could observe for MIP was the same as for Lisbon, that weights $(6, 4, 2)$ produced worse runtimes than $(3, 2, 1)$.

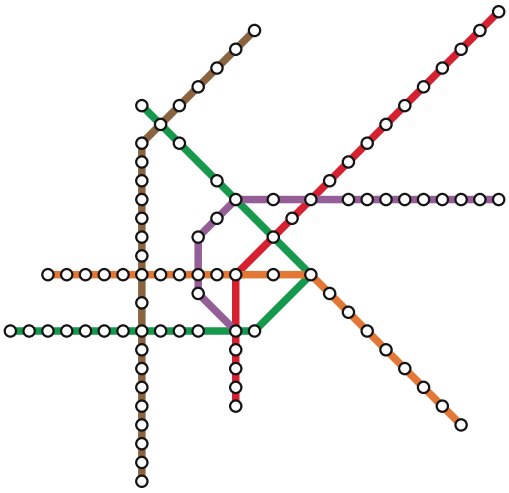
For the metro map of Vienna we could not observe the runtime discrepancies between the weight vectors. All solvers found an optimal solution within the given runtime limit. The only discrepancies were the known ones for SMT that runtimes for weights $(3, 2, 1)$ were better than for weights $(6, 4, 2)$. Runtimes with the MIP solver were significantly faster than with SAT and SMT.

5.5 Solution quality

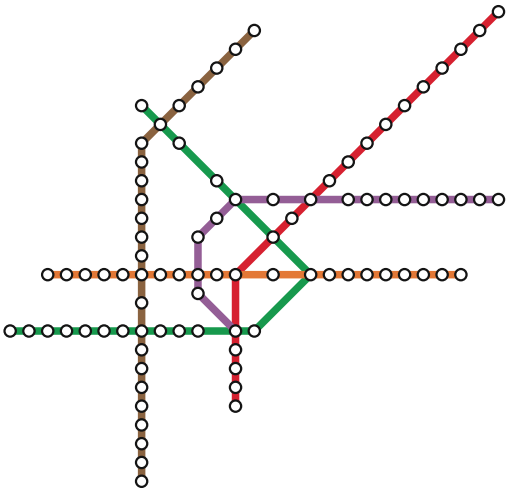
When talking about the quality of a solution the indicator is the cost value of the solution. It is calculated as described in Chapter 2 by summing up line bends, deviations of sections and edge length each multiplied with the corresponding input weights. An optimal solution is a solution with a cost value lower or equal to all other possible solutions. It is not necessary that the optimal solution is unique, it is possible to have two solutions with the same cost value, that are both optimal. Solutions with the same cost value are equivalent with respect to the input weights. This is just our definition of optimality, so recall that there are other measures of solution quality, which were not optimized in the models and are not evaluated here.

This behavior can easily be observed on the metro map of Vienna. Throughout all experiments on instances of Vienna we found 4 different solutions that are given in Figure 5.2. For weights $(5, 2, 1)$ all three solvers found the same solution that is shown in Figure 5.2b with cost value 87. MIP and SAT produced the same solution for weights $(3, 2, 1)$ and $(6, 4, 2)$ with cost value 70 and 140 respectively. This solution is shown in Figure 5.2a Note that the cost values of optimal solutions for the same instance with weights $(3, 2, 1)$ and weights $(6, 4, 2)$ have to exactly differ by a factor of 2. SMT generated different solutions for weights $(3, 2, 1)$ and $(6, 4, 2)$. These two solutions are also different from the solutions of the other solvers and given in Figure 5.2c and Figure 5.2d. Nevertheless the solution for weights $(3, 2, 1)$ still has cost vale 70 while the solution for weights $(6, 4, 2)$ has cost value 140 respectively.

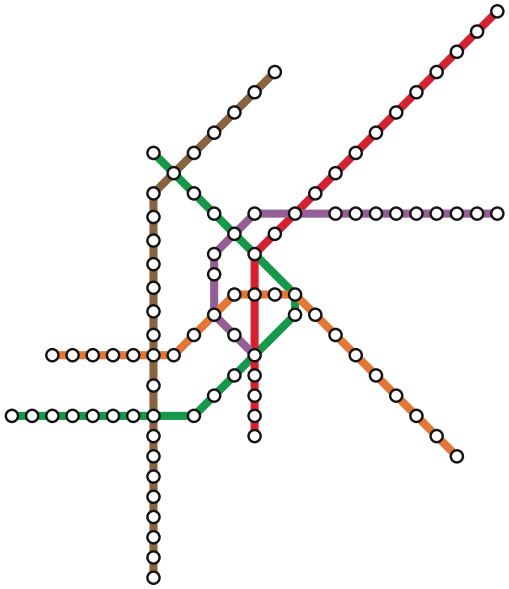
Comparing the quality of the solutions found in the carried out experiments does not make sense for most of the instances, because optimal solutions were found by all solvers



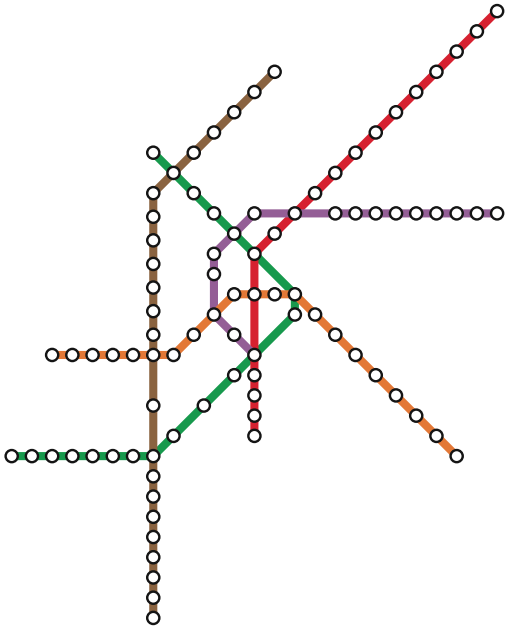
(a) MIP and SAT: $(3, 2, 1)$ and $(6, 4, 2)$.



(b) MIP, SAT and SMT: $(5, 2, 1)$.



(c) SMT: $(3, 2, 1)$.



(d) SMT: $(6, 4, 2)$.

Figure 5.2: Different solutions found for Vienna.

and are equivalent with respect to the used weights. The only meaningful instances to compare solution quality are the instances that produced timeouts. Primary these are the instances for the Karlsruhe metro map. Runs on other instances that reached the runtime limit did still find the optimal solution.

city	f1	f2	f3	algo	cost	diff	%
Karlsruhe	3	2	1	mip	237		
				smt	497	+260	+109,7%
	5	2	1	mip	350		
				smt	585	+235	+67,1%
	6	4	2	mip	474		
				smt	990	+516	+108,9%

Table 5.4: Comparison of cost values between MIP and SMT for Karlsruhe.

A comparison of the solutions found on the Karlsruhe instances is given in Table 5.4. Again as before the first five columns specify the concrete instance. Then the column `cost` gives the cost of the found solutions. As stated beforehand all 20 runs on each of these instances produced the same solutions for the same instance. The last two columns `diff` and % give the absolute and relative difference between the solutions found by the MIP and the SMT solver for each input. The results show that the MIP solver found significantly better solutions for each instance. This proves that the solutions found with SMT cannot be optimal. We omitted the solutions of SAT in this table, because SAT did not produce solutions for Karlsruhe in general.

Overall our results indicate that the MIP implementation is faster then our approaches with SAT and SMT. For finished instances MIP had faster runtimes in most cases and was not significantly slower for the exceptions. For instances that produced timeouts without finding optimal solutions MIP found significantly better solutions than SAT and SMT. Comparing SAT and SMT does not give a clear answer which approach is better, because the results notably depend on the input instances. For timed out instances we cannot say if SAT would have produced optimal or better solutions than SMT because of the lack of intermediate solutions. The Montreal instances gave indications that SMT has longer initialization overheads then the other solvers. The instances with the second shortest runtimes were the Washington instances. This is also the second instance on which SMT performed significantly worse than SAT which could confirm the theory of initialization overhead on SMT further. It could be a sign that the SMT encoding works better on bigger instances and SAT is better for smaller instances.

CHAPTER 6

Conclusion

The main contribution of this thesis are the SAT and the SMT encodings of the schematic metro map problem. Both encodings are developed from the MIP encoding by Nöllenburg and Wolff [NW10]. The main task was to encode the linear inequality constraints in boolean variables and logical formulas. The efficiency of these encodings was tested with different real world instances on state of the art SAT and SMT solvers. We compared the runtimes and solution quality with results of the MIP encoding on the same instances. The experiments for all solvers and encodings were executed on the same conditions, since we used the same cluster nodes and the same input for the experiments.

The runtimes with SAT and SMT were not competitive enough to beat the MIP implementation. This indicates that it is not reasonable to prefer SAT or SMT over MIP for the schematic metro map problem, but this is not guaranteed. We know that MIP solvers are designed to use callback functions and add additional constraints on demand. This is not supported in most of the available SAT or SMT solvers. The SAT solver we used did also not have this feature at all, while the SMT solver implemented an experimental version of it. Generally SAT solvers were primarily designed to solve decision problems instead of optimization problems and the same holds for SMT. Hence there is a chance that the results of our encodings can get significantly better with further developed SAT or SMT solvers.

The comparison between SAT and SMT did not show a clear preference so we can not clearly state which is the better approach. We had input instances on which SAT performed significantly better than SMT and other instances with results the other way round. Something that we observed about SMT was that it seem like there is a large overhead for small instances or instances with shorter runtimes. One good example for this behaviour were the Montreal instances. This indicates that SMT has a larger overhead for initializing the model in the first hand, independent of the input size and the remaining computation time.

An open area to discuss is the existing set of design criteria. As mentioned earlier we worked specifically on the design rules proposed by Nöllenburg and Wolff [NW10]. It is worth an analysis which other sets of criteria are possible to implement with SAT and SMT. In fact we already tried to implement the design rules given by Ovenden [Ove08] or Roberts [Rob14a, Rob14b] in rudiments and it did work quite well. They do not allow bends in interconnect stations but bends on edges between stations are allowed. These rules can easily be implemented without significant changes in our model even if it is not possible to have bends in the present model. Instead some preprocessing steps are needed. We added dummy stations on edges that are deleted during postprocessing. To do so we used the fact that the used version of the deg-2 heuristic keeps one or two stations on each path and modified the implementation such that these stations are marked as dummy stations. For paths of vertices of degree two with less stations on it we add one or two dummy stations. The deleted stations are added during postprocessing such that they are not placed on a line bend to follow the design rule. The small modification that we had to make on our model was to do not allow bends on all stations, but only on dummy stations. With these steps we could easily produce schematic metro maps that follow the design rules of Ovenden [Ove08] or Roberts [Rob14a, Rob14b].

Another modification that can be worked on is an improvement of the minimal edge length. At the moment we only have constraints to give an edge a minimal length, independent of the other edge lengths. It would make sense to introduce groups of edges and state that the summed edge length of the group should have a minimal length. For example this could be used for edges modified by the deg-2 heuristic. With unary encoding in SAT it could be more difficult to implement sums over several edge lengths, but with SMT it should be quite easy to add sums over a group of variables.

Last but not least our SAT encoding could be improved by using any other integer encoding instead of unary encoding. For example bit vectors implementing binary encoding could be a possibility that maybe leads to more efficient results.

List of Figures

List of Tables

Bibliography

[BBS20] Hannah Bast, Patrick Brosi, and Sabine Storandt. Metro maps on octilinear grid graphs. *Computer Graphics Forum*, 39(3):357–367, 2020.

[BBS21] Hannah Bast, Patrick Brosi, and Sabine Storandt. Metro maps on flexible base grids. In *17th International Symposium on Spatial and Temporal Databases, SSTD 2021*, pages 12–22. ACM, 2021.

[Bec33] Harry Charles Beck. London tube map. *London Transport Museum*, 1933.

[BFT16] Clark Barrett, Pascal Fontaine, and Cesare Tinelli. The Satisfiability Modulo Theories Library (SMT-LIB). `www.SMT-LIB.org`, 2016.

[BSST21] Clark W. Barrett, Roberto Sebastiani, Sanjit A. Seshia, and Cesare Tinelli. Satisfiability modulo theories. In *Handbook of Satisfiability - Second Edition*, pages 1267–1329. IOS Press, 2021.

[Che22] Otfried Cheong. The ipe extensible drawing editor. `https://ipe.otfried.org/`, 1993–2022. Accessed: 2022-08-24.

[CR14] Daniel Chivers and Peter Rodgers. Octilinear force-directed layout with mental map preservation for schematic diagrams. In *Diagrammatic Representation and Inference - 8th International Conference, Diagrams 2014*, pages 1–8. Springer, 2014.

[DHM08] Tim Dwyer, Nathan Hurst, and Damian Merrick. A fast and simple heuristic for metro map path simplification. In *International Symposium on Visual Computing*, pages 22–30. Springer, 2008.

[DM47] Augustus De Morgan. *Formal Logic, or, The Calculus of Inference, Necessary and Probable*. Taylor and Walton, London, 1847.

[dMBWN] Leonardo de Moura, Nikolaj Bjorner, Christoph M. Wintersteiger, and Lev Nachmanson. Z3Prover. `https://github.com/Z3Prover/z3`. Accessed: 2022-08-24.

[DvKSW12] Kasper Dinkla, Marc J. van Kreveld, Bettina Speckmann, and Michel A. Westenberg. Kelp diagrams: Point set membership visualization. *Computer Graphics Forum*, 31(3pt1):875–884, 2012.

[ES06] Niklas Eén and Niklas Sörensson. Translating Pseudo-Boolean Constraints into SAT. *Journal on Satisfiability, Boolean Modeling and Computation*, 2(1-4):1–26, January 2006.

[Eul58] Leonhard Euler. Elementa doctrinae solidorum. *Novi commentarii academiae scientiarum Petropolitanae*, pages 109–140, 1758.

[Fár48] István Fáry. On straight-line representation of planar graphs. *Acta Scientiarum Mathematicarum*, 11:229–233, 1948.

[GT01] Ashim Garg and Roberto Tamassia. On the computational complexity of upward and rectilinear planarity testing. *SIAM Journal on Computing*, 31(2):601–625, 2001.

[Her64] Israel Nathan Herstein. Topics in algebra. *Waltham: Blaisdell Publishing Company*, 1964.

[HMdN06] Seok-Hee Hong, Damian Merrick, and Hugo A. D. do Nascimento. Automatic visualisation of metro maps. *Journal of Visual Languages and Computing*, 17(3):203–224, 2006.

[IMM18] Alexey Ignatiev, António Morgado, and João Marques-Silva. Pysat: A python toolkit for prototyping with SAT oracles. In *International Conference on Theory and Applications of Satisfiability Testing, SAT 2018*, pages 428–437. Springer, 2018.

[JWKN21] Ben Jacobsen, Markus Wallinger, Stephen G. Kobourov, and Martin Nöllenburg. Metrosets: Visualizing sets as metro maps. *IEEE Transactions on Visualization and Computer Graphics*, 27(2):1257–1267, 2021.

[LCJ19] Bjørnar Luteberget, Koen Claessen, and Christian Johansen. Automated drawing of railway schematics using numerical optimization in SAT. In *International Conference on Integrated Formal Methods*, pages 341–359. Springer, 2019.

[Leg85] Adrien-Marie Legendre. *Eléments de géométrie*. Manceaux, 1885.

[McC78] John McCarthy. History of LISP. In Richard L. Wexelblat, editor, *History of Programming Languages*, pages 173–185. ACM, 1978.

[MDM14] António Morgado, Carmine Dodaro, and João Marques-Silva. Core-guided maxsat with soft cardinality constraints. In *International Conference on Principles and Practice of Constraint Programming, CP 2014*, pages 564–573. Springer, 2014.

[MG06] Damian Merrick and Joachim Gudmundsson. Path simplification for metro map layout. In *International Symposium on Graph Drawing*, pages 258–269. Springer, 2006.

[MHL+13] António Morgado, Federico Heras, Mark H. Liffiton, Jordi Planes, and João Marques-Silva. Iterative and core-guided maxsat solving: A survey and assessment. *Constraints - An International Journal*, 18(4):478–534, 2013.

[MIM14] António Morgado, Alexey Ignatiev, and João Marques-Silva. MSCG: robust core-guided maxsat solving. *Journal on Satisfiability, Boolean Modeling and Computation*, 9(1):129–134, 2014.

[NN20] Soeren Nickel and Martin Nöllenburg. Towards data-driven multilinear metro maps. In *International Conference on Theory and Application of Diagrams*, pages 153–161. Springer, 2020.

[Nöl05] Martin Nöllenburg. Automated drawing of metro maps. Master's thesis, Universität Karlsruhe, Fakultät für Informatik, 2005.

[NW10] Martin Nollenburg and Alexander Wolff. Drawing and labeling high-quality metro maps by mixed-integer programming. *IEEE Transactions on Visualization and Computer Graphics*, 17(5):626–641, May 2010.

[Ove08] Mark Ovenden. *Paris Metro style in map and station design*. Capital Transport Pub., 2008.

[Pat13] Maurizio Patrignani. Planarity testing and embedding. In Roberto Tamassia, editor, *Handbook on Graph Drawing and Visualization*, pages 1–42. Chapman and Hall/CRC, 2013.

[Pfa10] Bernhard Pfahringer. Conjunctive normal form. In Claude Sammut and Geoffrey I. Webb, editors, *Encyclopedia of Machine Learning*, pages 209–210. Springer, 2010.

[Rob14a] Maxwell J Roberts. Schematic maps in the laboratory. *Schematic Mapping Workshop*, 2014.

[Rob14b] Maxwell J Roberts. What's your theory of effective schematic map design? *Schematic Mapping Workshop*, 2014.

[SR34] Ernst Steinitz and Hans Rademacher. *Vorlesungen über die Theorie der Polyeder*. Julius Springer, 1934.

[SR04] Jonathan M. Stott and Peter Rodgers. Metro map layout using multicriteria optimization. In *8th International Conference on Information Visualisation, IV 2004*, pages 355–362. IEEE Computer Society, 2004.

[Ste51] Sherman K. Stein. Convex maps. *Proceedings of the American Mathematical Society*, 2(3):464–466, 1951.

[Tam13] Roberto Tamassia. *Handbook on Graph Drawing and Visualization*. Chapman and Hall/CRC, 2013.

[vDL18] Thomas C. van Dijk and Dieter Lutz. Realtime linear cartograms and metro maps. In *Proceedings of the 26th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*, pages 488–491. ACM, 2018.

[Vis13] Luca Vismara. Planar straight-line drawing algorithms. In Roberto Tamassia, editor, *Handbook on Graph Drawing and Visualization*, pages 193–222. Chapman and Hall/CRC, 2013.

[VR20] Guido Van Rossum. *The Python Library Reference, release 3.6.9*. Python Software Foundation, 2020.

[Wag36] Klaus Wagner. Bemerkungen zum vierfarbenproblem. *Jahresbericht der Deutschen Mathematiker-Vereinigung*, 46:26–32, 1936.

[Wal] Markus Wallinger. Ipey python package. `https://github.com/mwallinger-tu/ipey`. Accessed: 2022-08-24.

[WC11] Yu-Shuen Wang and Ming-Te Chi. Focus+context metro maps. *IEEE Transactions on Visualization and Computer Graphics*, 17(12):2528–2535, December 2011.

[WNT+20] Hsiang-Yun Wu, Benjamin Niedermann, Shigeo Takahashi, Maxwell J. Roberts, and Martin Nöllenburg. A survey on transit map layout – from design, machine, and human perspectives. *Computer Graphics Forum*, 39(3):619–646, 2020.

[YDG+16] Vahan Yoghourdjian, Tim Dwyer, Graeme Gange, Steve Kieffer, Karsten Klein, and Kim Marriott. High-quality ultra-compact grid layout of grouped networks. *IEEE Transactions on Visualization and Computer Graphics*, 22(1):339–348, January 2016.